



Data Structures

Jasraj Meena

Assistant Professor

Department of Information Technology

Delhi Technological University



Tree Data Structures

- **Trees [1]:**

- Data Frequently contain a hierarchical relationship between various elements example: records, family tree, table of contents. The data structure which reflects this relationship is called a tree or rooted tree graph.

- **Example:**

- Consider a algebraic Expression

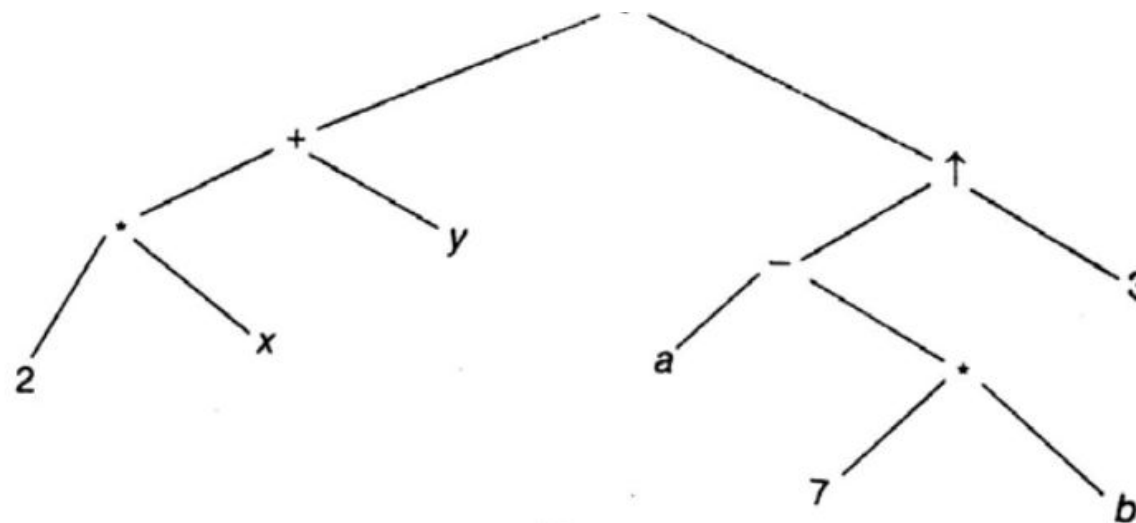
$$(2x + y)(a - 7b)^3$$

Tree Data Structures

- **Trees [1]:**

- **Example:** Consider a algebraic Expression $(2x + y)(a - 7b)^3$

Using the vertical a arrow for exponentiation and asterisk (*) for multiplication,



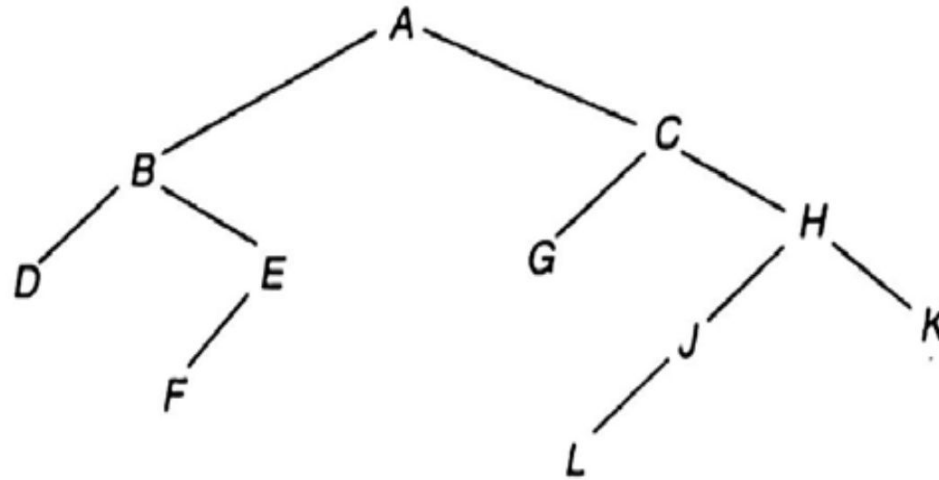


Binary Tree [1]

- A binary tree T is defined as a **finite set of elements**, called nodes, such that:
 - T is **empty** (called the **null tree** or empty tree), or
 - T contains a distinguished node R , called the root of T , and the remaining nodes of T form an ordered pair of disjoint binary trees T_1 and T_2 .
- If T does contain a root R , then the two trees T_1 and T_2 are called, respectively, the **left and right subtree** of R .
- If T_1 is nonempty, then its root is called the **left successor** of R ; similarly, if T_2 is nonempty, then its root is called the right successor of R .

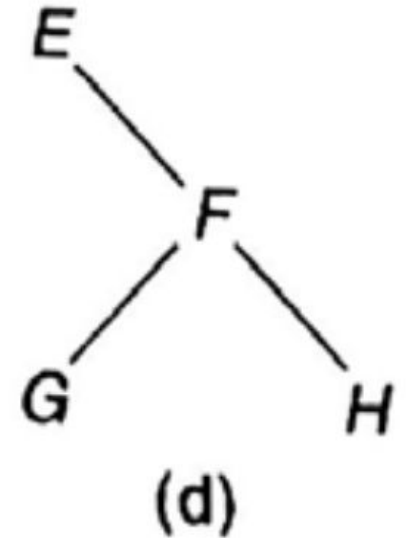
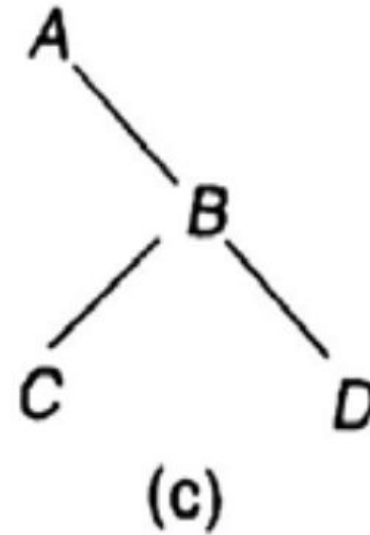
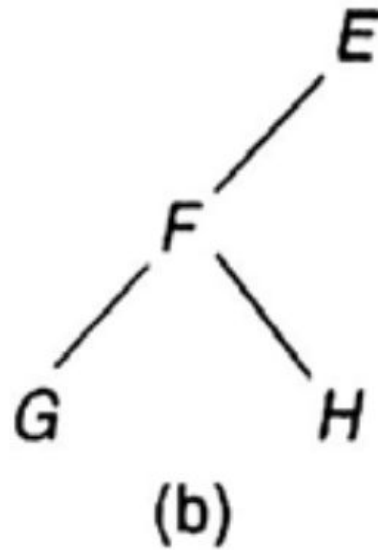
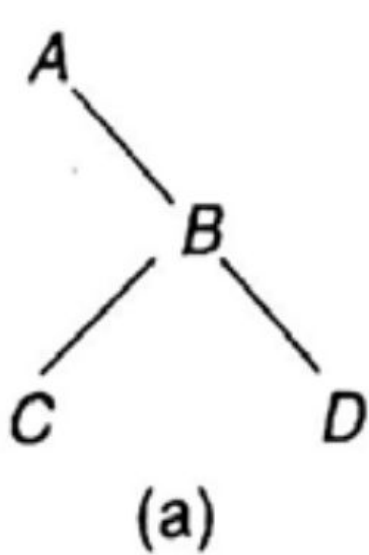
Binary Tree [1]

- A binary tree T is defined as a **finite set of elements**, called nodes. Example:
- Any node n in a binary tree T has either 0, 1, 2 successors.
- If a node is a terminal node (leaf node), then both its left and right subtree are empty.



Binary Tree [1]

- Which one is copied or similar?



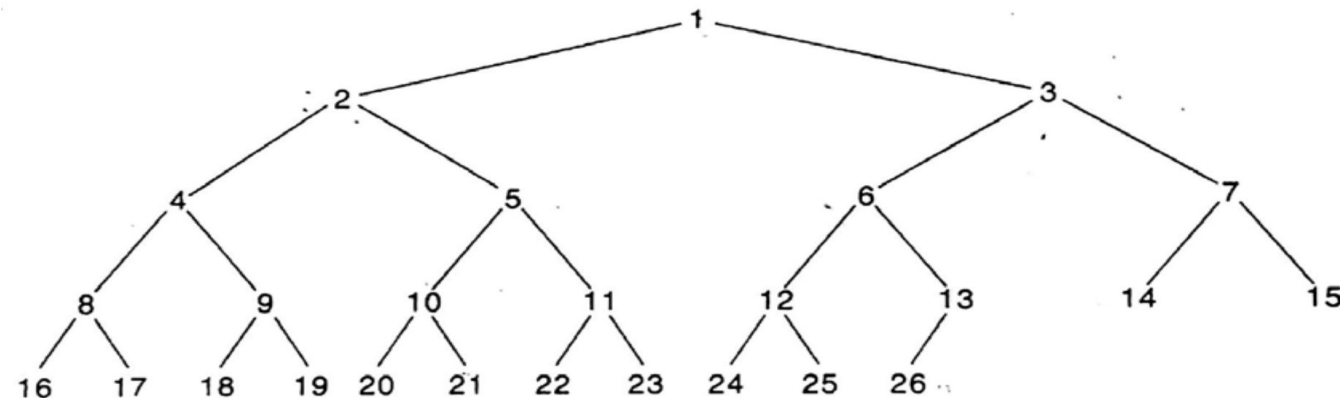


Terminology [1]

- Terminology describing family relationships is frequently used to describe relationship between nodes of a tree T.
 - Root
 - Parent
 - Child
 - Predecessor
 - Descendant and ancestor
 - Leaf node
 - Level number
 - Depth or height of the tree
- Binary tree T and T1 are said to be **similar** if they have **same structure** or, in other words, if they have the same shape. The trees are said to be **copies** if they are similar and if they have the **same contents at corresponding nodes**.

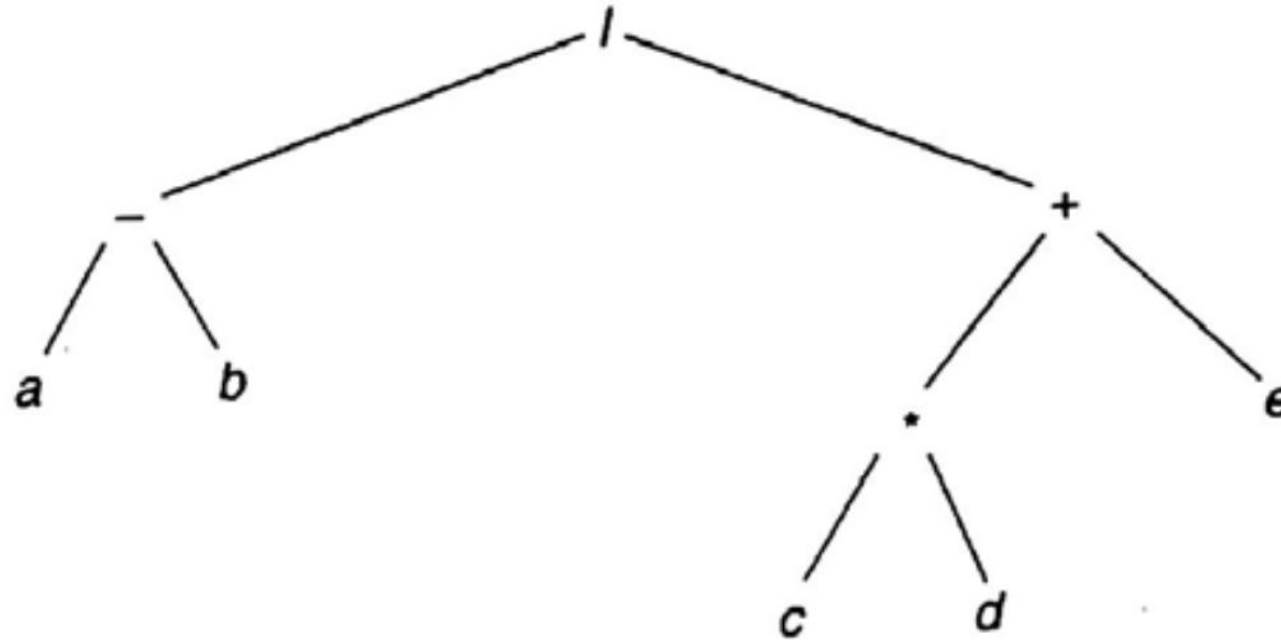
Complete Binary Tree [1]

- Consider any binary tree T . each node of T can have at most two children. Accordingly, one can show that level r of T can have at most 2^r nodes.
- The tree T is said to be complete if all its levels, except possibly the last, have the maximum number of possible nodes, and if all the nodes at the last level appear as far left as possible. Example:



Complete Binary Tree [1]

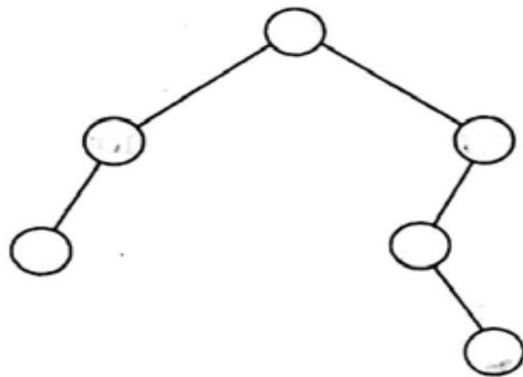
- Example: Check whether given tree is complete binary tree or not?



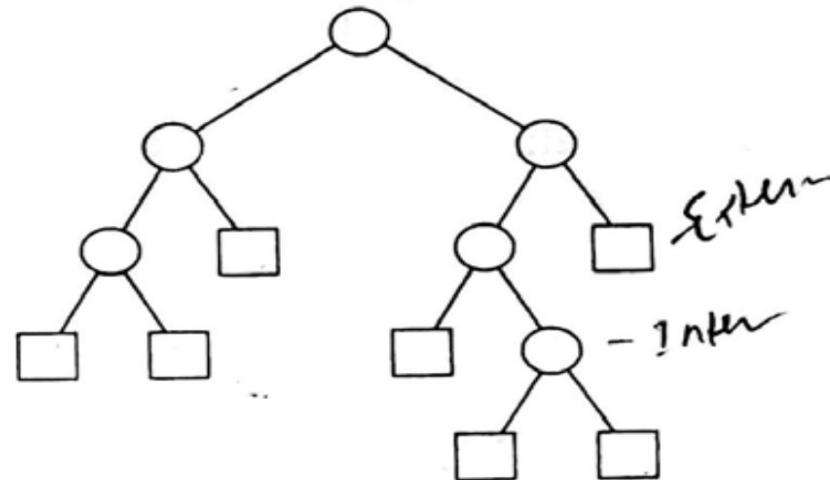
Extended Binary Tree: 2 tree [1]



- A binary tree T is said to be a 2-tree or extended binary tree if each node N has either 0 or 2 children. Nodes with 2 children is called internal node and nodes with 0 children is called external nodes.



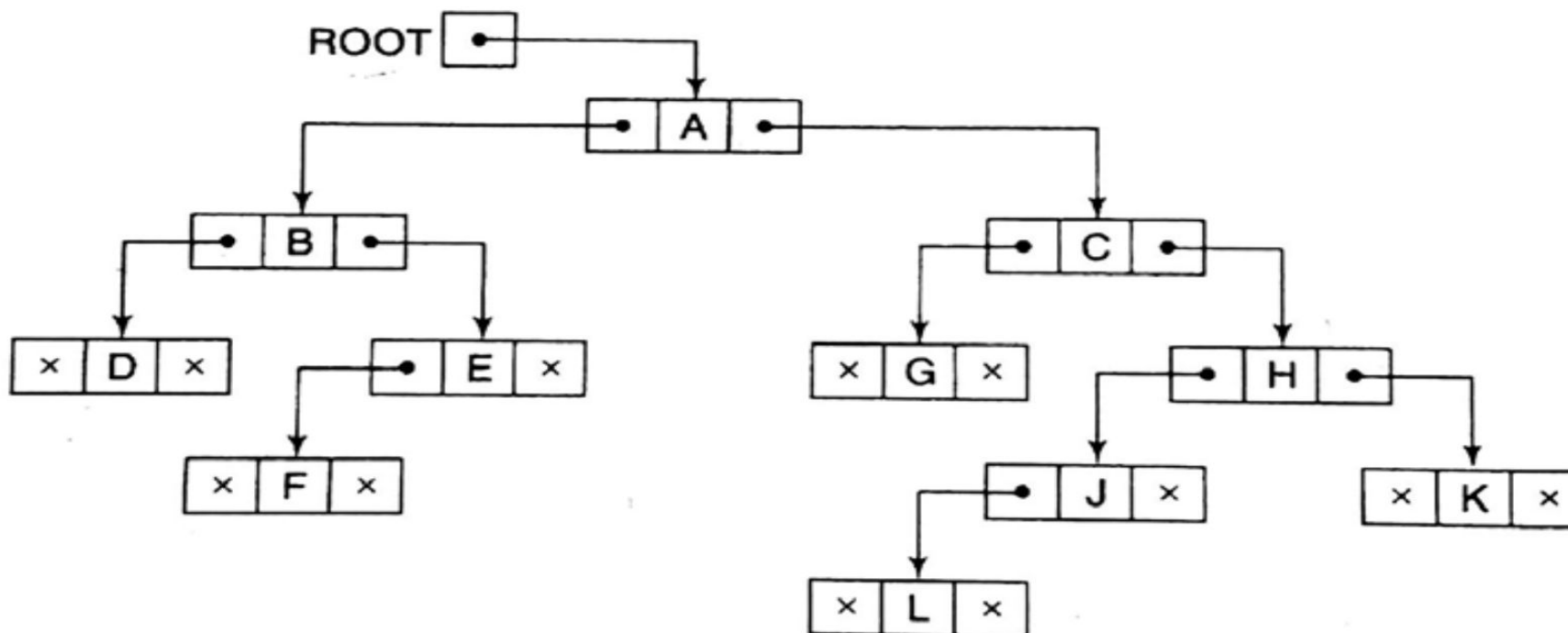
(a) Binary tree T



(b) Extended 2-tree

Representation of Binary Tree [1]

- **Linked List Representation:**





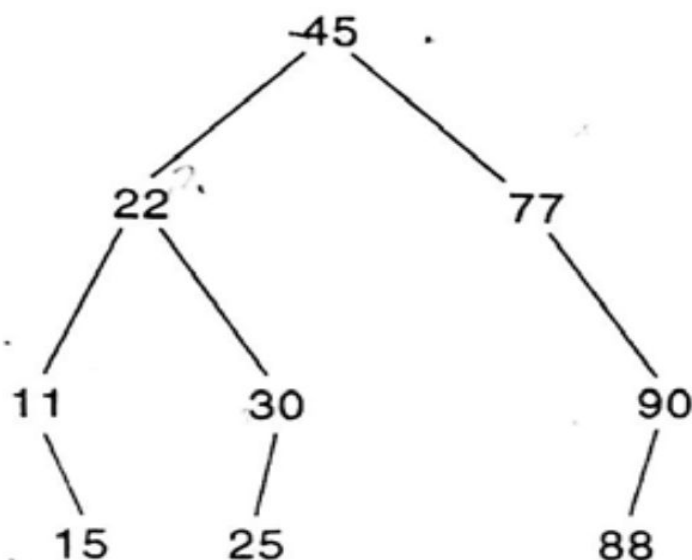
Representation of Binary Tree [1]

- **Sequential Representation:** Suppose T is a binary tree that is complete or nearly complete. The representation uses only a single linear array tree as follows:

- (a) The root R of T is stored in $TREE[1]$.
- (b) If a node N occupies $TREE[K]$, then its left child is stored in $TREE[2 * K]$ and its right child is stored in $TREE[2 * K + 1]$.

Representation of Binary Tree [1]

- **Sequential Representation:** Example



	TREE
1	45
2	22
3	77
4	11
5	30
6	
7	90
8	
9	15
10	25
11	
12	
13	
14	88
15	
16	
...	
29	



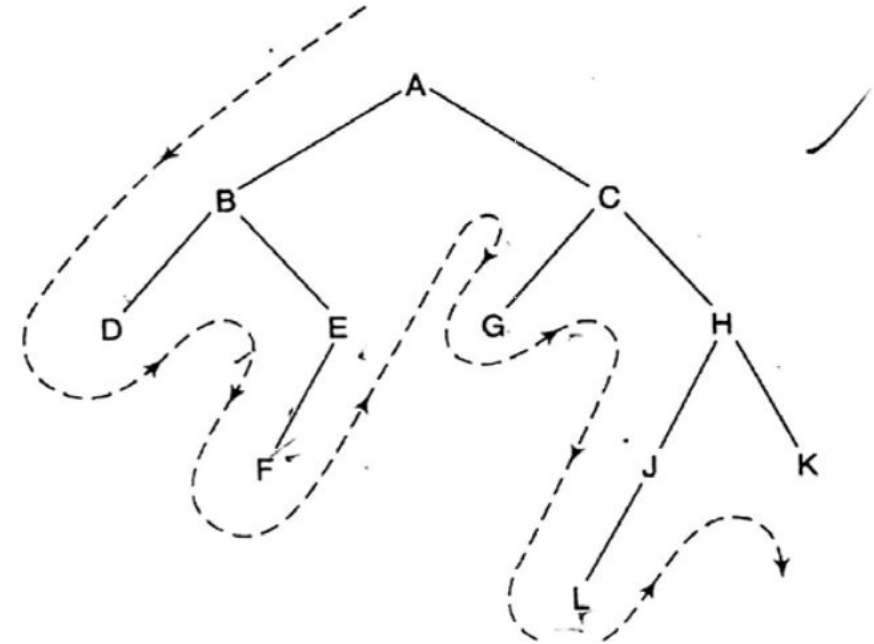
Traversing Binary Tree

- ❖ **Tree ways of traversing Binary Tree**
 - ❖ **Pre-order traversal**
 - ❖ **In-order traversal**
 - ❖ **Post-order traversal**

Traversing Binary Tree [1]

❖ Pre-order traversal:

- ❖ Process the root node R.
- ❖ Traverse the left subtree of R in pre-order.
- ❖ Traverse the right subtree of R in pre-order.





Traversing Binary Tree [1]

❖ In-order traversal:

- ❖ Traverse the left subtree of R in In-order.
- ❖ Process the root node R.
- ❖ Traverse the right subtree of R in In-order.

❖ Post-order traversal:

- ❖ Traverse the left subtree of R in Post-order.
- ❖ Traverse the right subtree of R in Post-order.
- ❖ Process the root node R.



Traversing Binary Tree [1]

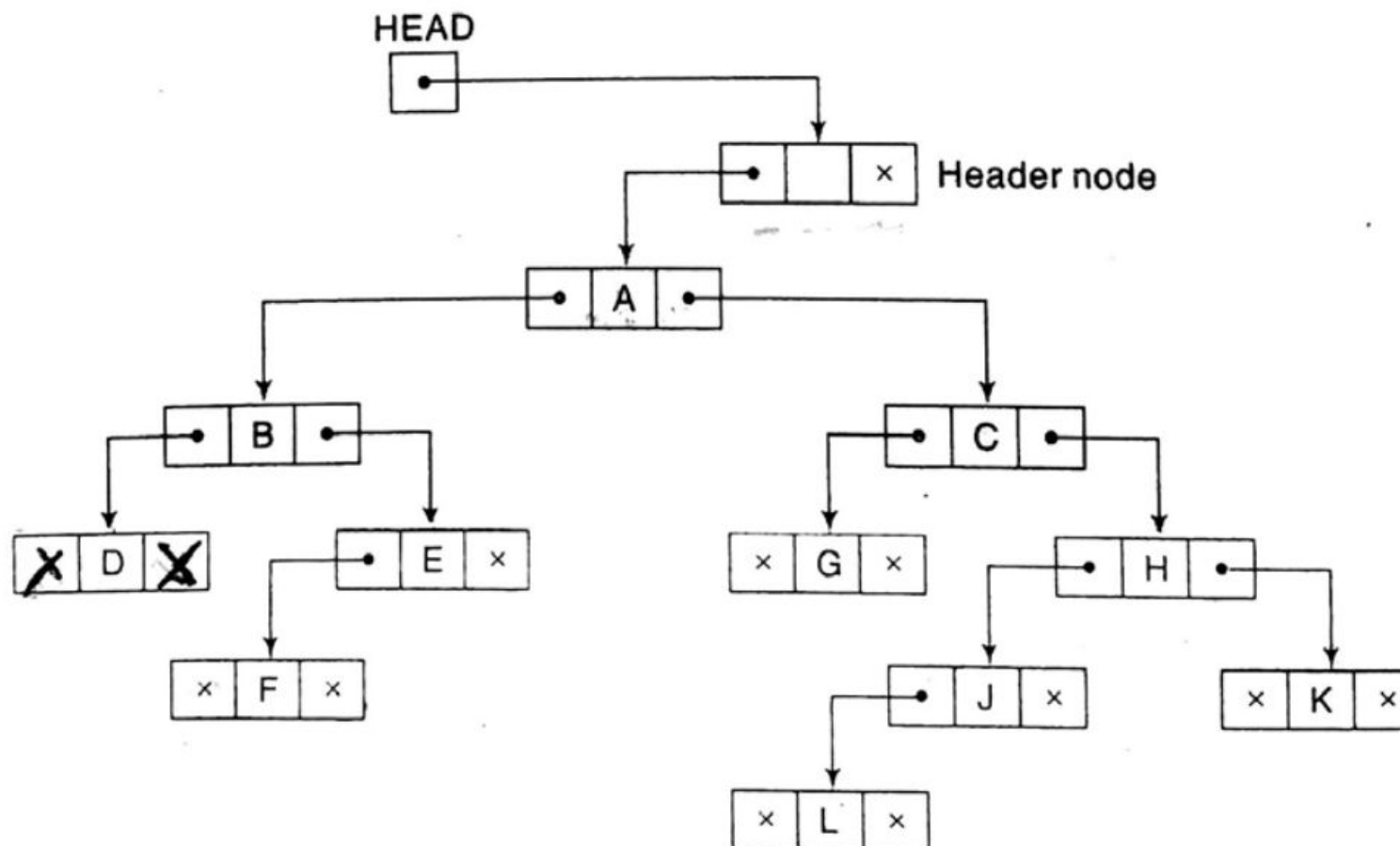
❖ In-order traversal:

- ❖ Traverse the left subtree of R in In-order.
- ❖ Process the root node R.
- ❖ Traverse the right subtree of R in In-order.

❖ Post-order traversal:

- ❖ Traverse the left subtree of R in Post-order.
- ❖ Traverse the right subtree of R in Post-order.
- ❖ Process the root node R.

Header Nodes; Threads [1]



Threads; Inorder Threading [1]



- ❖ Consider the **linked representation** of a binary tree T. Approximately half of the entries in the pointer fields LEFT and RIGHT will **contain null elements**.
- ❖ This space may be more efficiently used by **replacing the null entries** by some **other type of information**.
- ❖ Specifically, we will replace **certain null entries** by **special pointer which point to nodes higher in the tree**.
- ❖ These **special pointers** are called **threads**, and binary trees with such pointers are called **threaded trees**.

Threads; Inorder Threading [1]



- ❖ The threads in a threaded tree must be distinguished in some way from ordinary pointers.
- ❖ The threads in a diagram of a threaded tree are usually indicated by dotted lines.
- ❖ In computer memory, an extra 1-bit TAG field may be used to distinguish threads from ordinary pointers, or alternatively,
- ❖ Threads may be denoted by negative integers when ordinary pointers are denoted by positive integers.

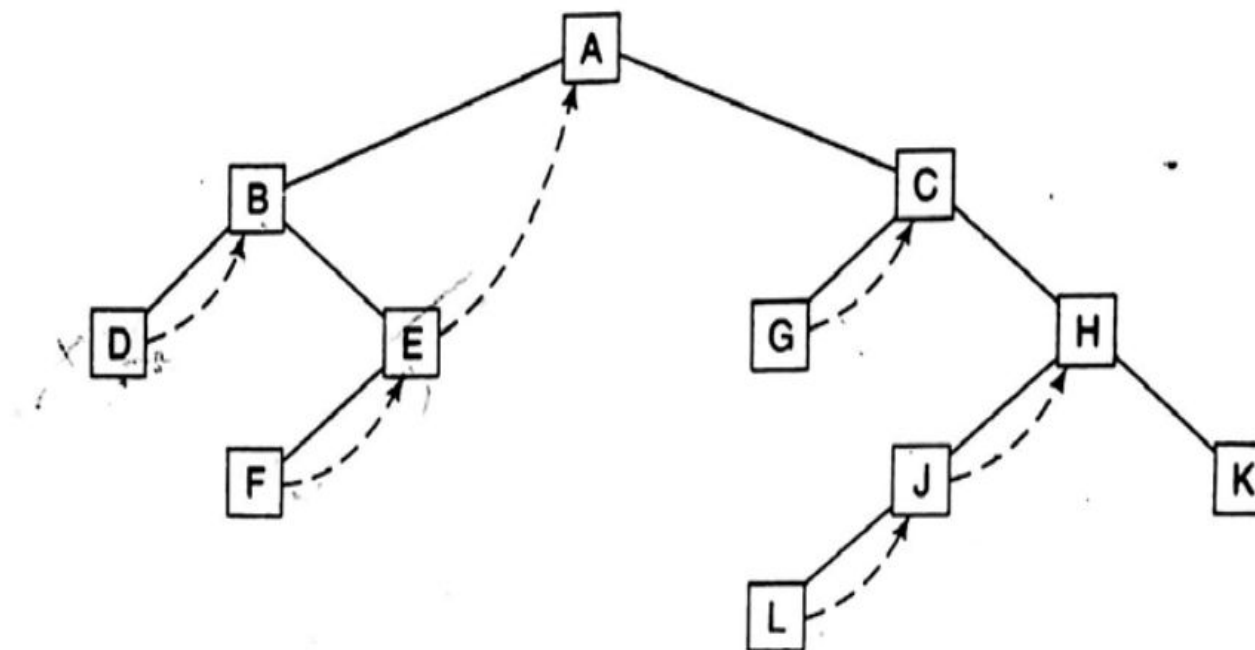
Threads; Inorder Threading [1]



- ❖ There are many ways to thread a binary tree T , but each threading will correspond to a particular traversal of T .
- ❖ Also, one may choose a one-way threading or a two-way threading.
- ❖ Unless otherwise stated, our threading will correspond to the in-order traversal of T .
- ❖ According, in the one-way threading of T , a thread will appear in the right field of a node and will point to the next node in the inorder traversal of T .

Threads; Inorder Threading [1]

- Example:



(a) One-way inorder threading

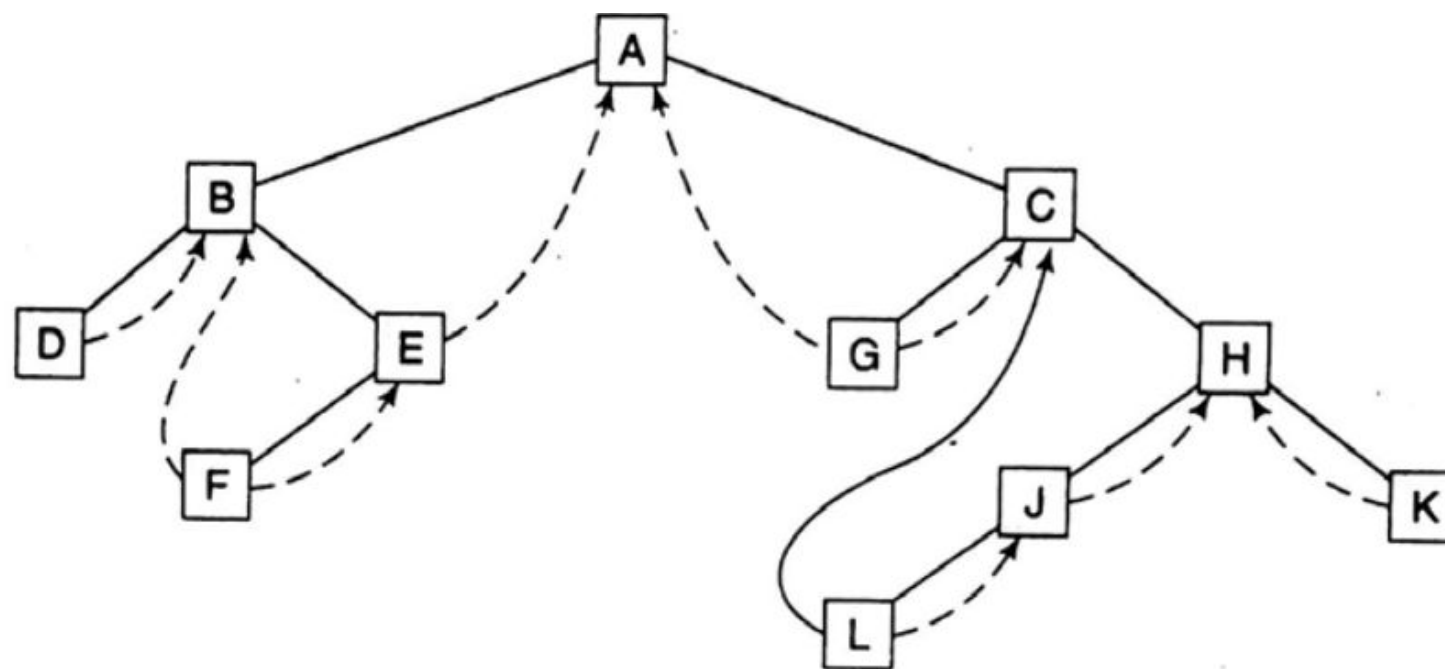
Threads; Inorder Threading [1]



- ❖ In **two-way threading** of T, a thread will also appear in **the LEFT field of a node** and will point to the **preceding node in the inorder traversal** of T.
- ❖ Furthermore, the left pointer of the first node and the right pointer of the last node (in the inorder traversal of T) will contain the null value when T does not have a header node, but will point to the header node when T does have a header node.

Threads; Inorder Threading [1]

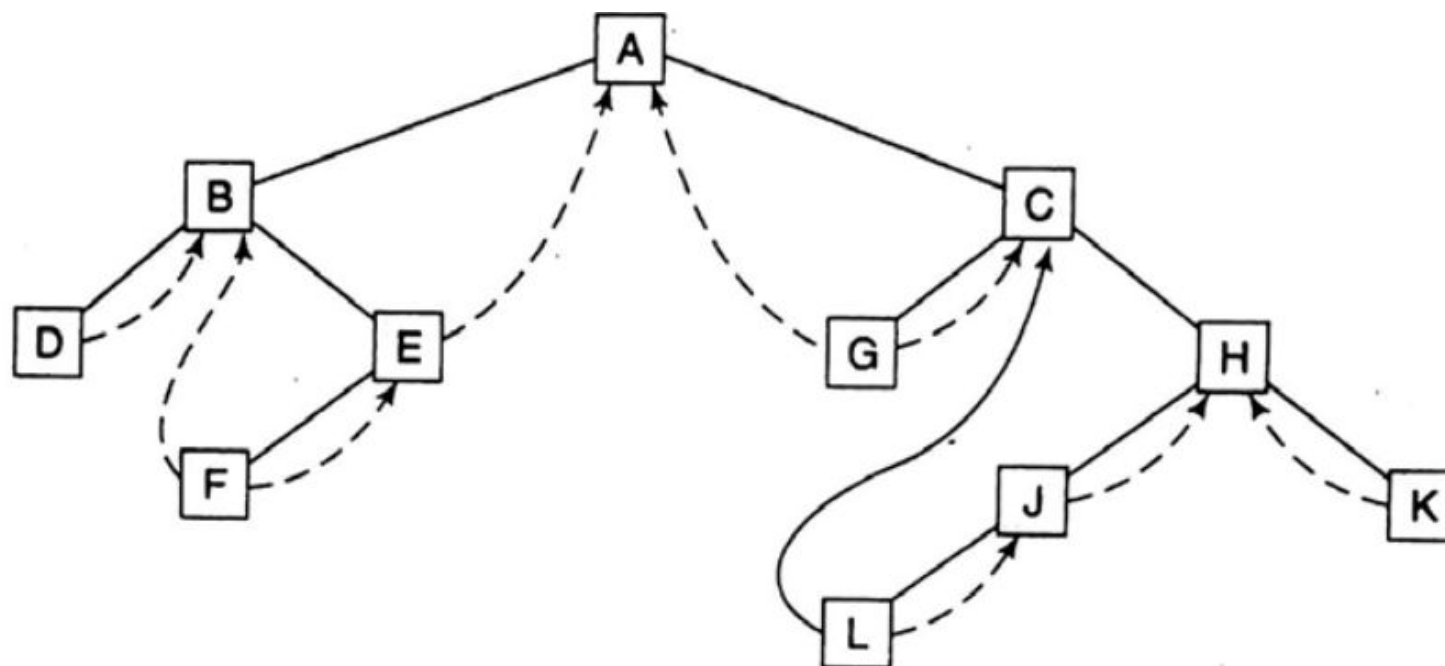
- Example:



(b) Two-way inorder threading

Threads; Inorder Threading[1]

- Example:



(b) Two-way inorder threading

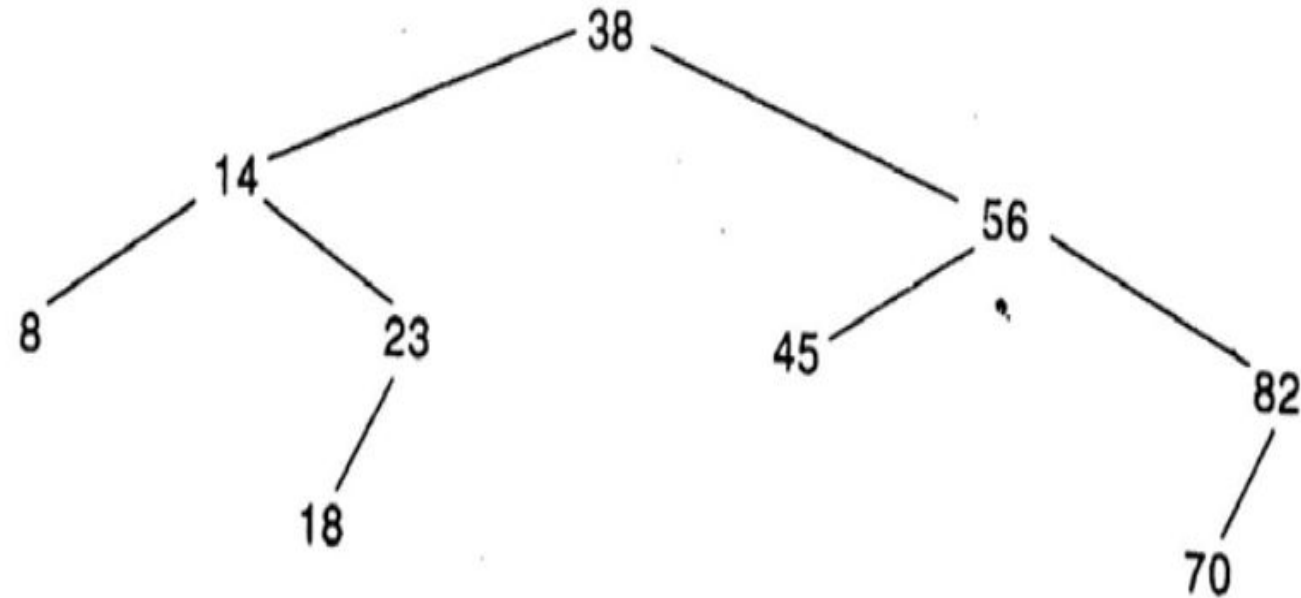


Binary Search Tree [1]

- ❖ Suppose T is a binary tree. Then T is called a **Binary Search Tree** (or binary sorted tree) if each node N of T has the following property:
 - ❖ The value at N is **greater than every value in the left subtree** of N and
 - ❖ Is **less than every value in the right subtree** of N .
- ❖ In **binary search tree** to search an element it will take **$O(\log n)$** time.
- ❖ It also enables one to easily **insert and delete elements**.

Binary Search Tree [1]

◆ Example:





Binary Search Tree [1]

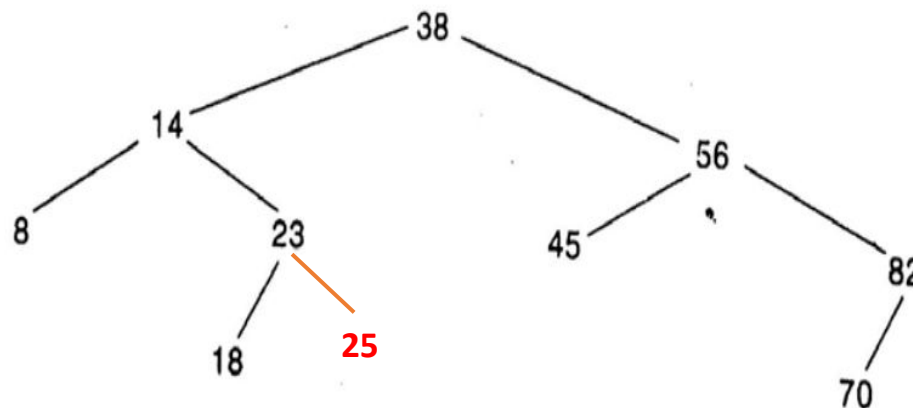
❖ Searching and Inserting in Binary Search Tree:

- ❖ Suppose an ITEM of information is given. The following algorithm finds the location of ITEM in a binary search tree T, or insert ITEM as a new node in its appropriate place in the tree:
 - ❖ Compare ITEM with the root node N of the Tree.
 - ❖ If $ITEM < N$, proceed to the left child of N
 - ❖ IF $ITEM > N$, proceed to the right child of N.
 - ❖ Repeat Step (a) until one of the following occurs:
 - ❖ We meet a node N such that $ITEM=N$. In this case the search is successful.
 - ❖ We meet an empty subtree, which indicates that the search is unsuccessful, and we insert ITEM in a place of the empty subtree.

Binary Search Tree [1]

❖ Searching and Inserting in Binary Search Tree:

- ❖ In other word, proceed from the root R, down through the tree until finding ITEM in T or inserting ITEM as a terminal node (leaf node) in T.
- ❖ Example: suppose ITEM=25, is given.





Problems based on Tree

-

Problem 1: Consider a collection of n data items, $A_1, A_2, A_3, \dots, A_n$ in a Binary Search Tree. Suppose we want to find and delete all duplicates in the collection. What will be the complexity of this problem?.



Binary Search Tree [1]

❖ Deleting in a Binary Search Tree:

- ❖ Suppose T is a binary search tree, and suppose an ITEM of information is given.
- ❖ In the deletion algorithm first we will find the location of the Node N which contains ITEM and also the location of the parent node $P(N)$. The way N is deleted from the tree depends primarily on the number of children of node N .



Binary Search Tree [1]

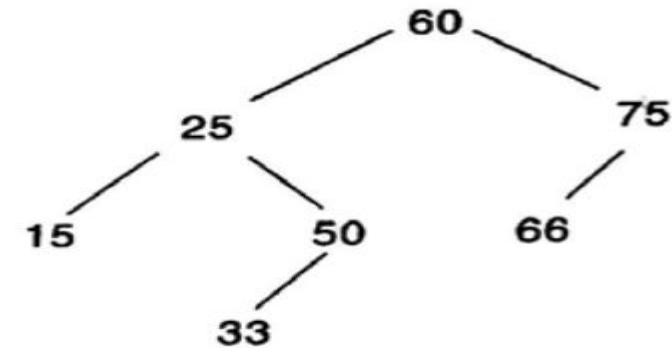
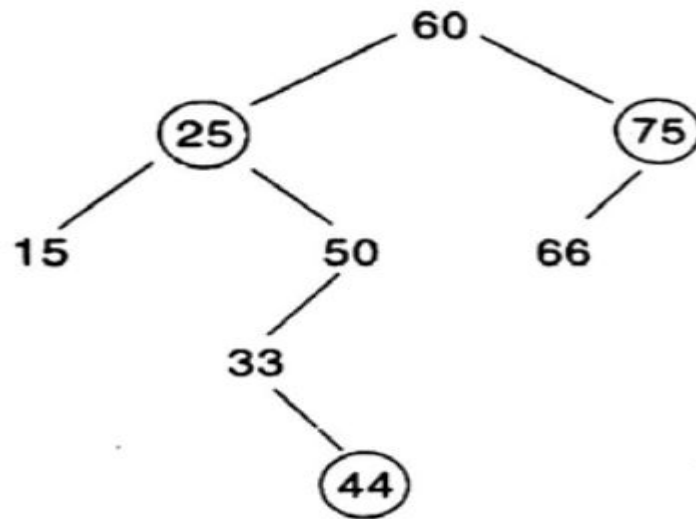
❖ Deleting in a Binary Search Tree:

- ❖ **Case 1:** N has no children. Then N is deleted from T by simply replacing the location of N in the parent node P(N) by the null pointer.
- ❖ **Case 2:** N has exactly one child. Then N is deleted from T by simply replacing the location of N in P(N) by the location of the only child of N.
- ❖ **Case 3:** N has two children. Let S(N) denote the inorder successor of N. The N is deleted from T by first deleting S(N) from T and the replacing node N in T by the node S(N)

Binary Search Tree : Deletion [1]



◆ Example:

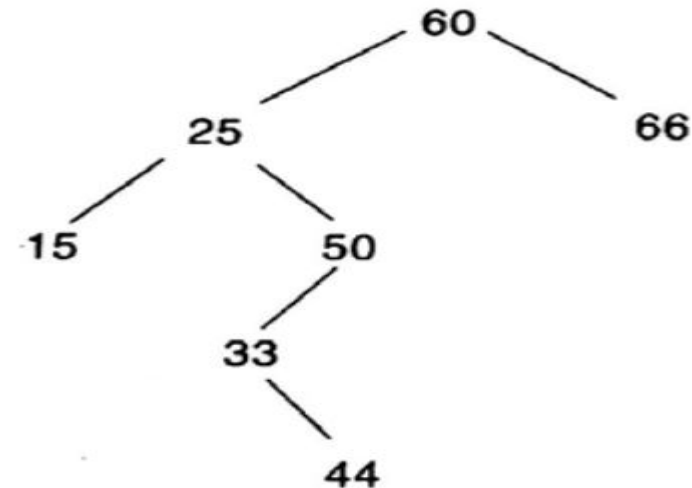
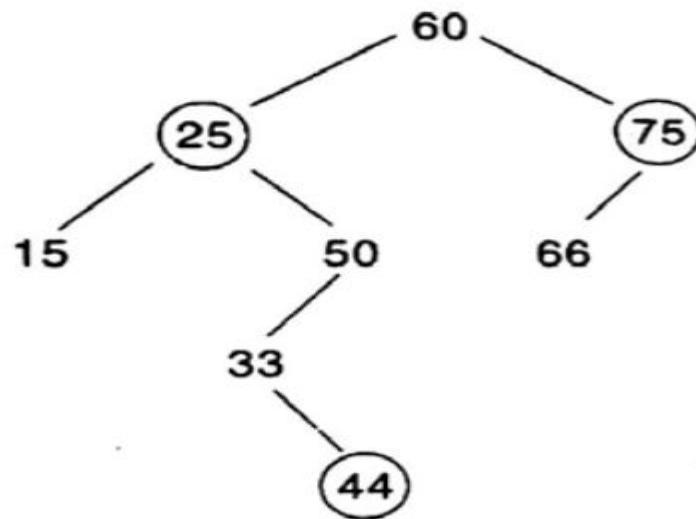


(a) Node 44 is deleted

Binary Search Tree : Deletion [1]



◆ Example:

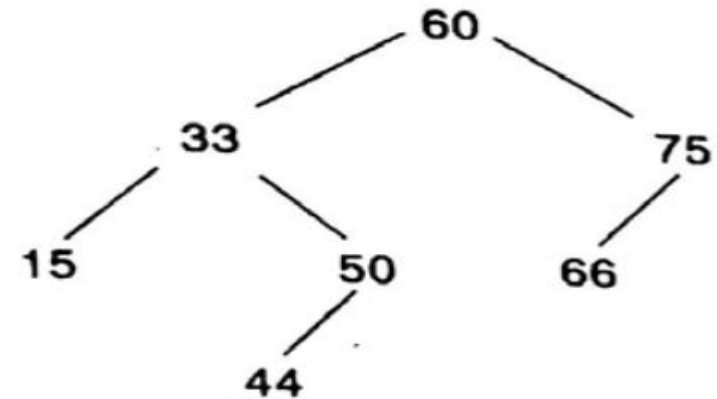
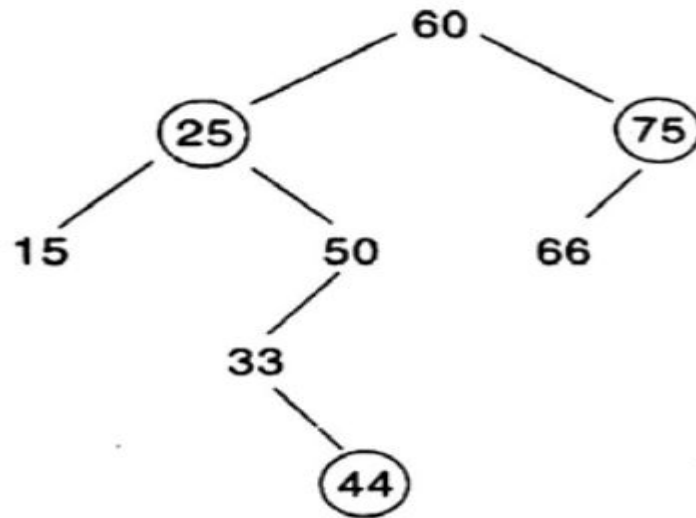


(a) Node 75 is deleted

Binary Search Tree : Deletion [1]



◆ Example:



(a) Node 25 is deleted



Problems based on Tree

Problem 2: Write a Algorithm to check if a given binary tree is a binary search tree or not?

Problems based on Tree



Problem 3: Write a Algorithm to count the number of leaves in a tree?

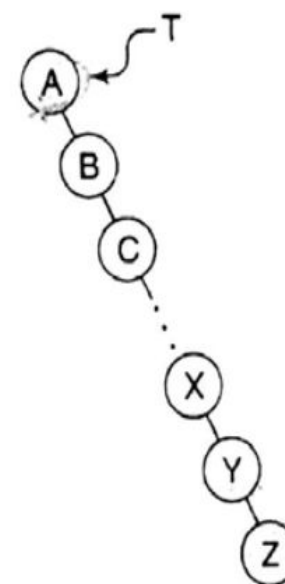
Problems based on Tree



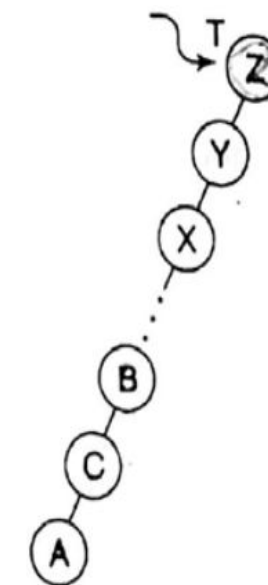
Problem 4: Write a Algorithm to create a mirror copy of a tree (left nodes become right and right nodes become left)!

AVL Search Tree [1]

- Consider the element A, B, C, D, ..., Z to be inserted into a binary search tree. Observe how the binary search tree turns out to be right skewed.
- Again the insertion of elements Z, Y, X, ..., C, B, A in that order in the binary search tree results in a left skewed binary search tree.
- The disadvantage of a skewed binary search tree is that the worst case time complexity of a search is $O(n)$.
- Therefore there arises the need to maintain the binary search tree to be of balanced height.



Right skewed binary search tree



Left skewed binary search tree

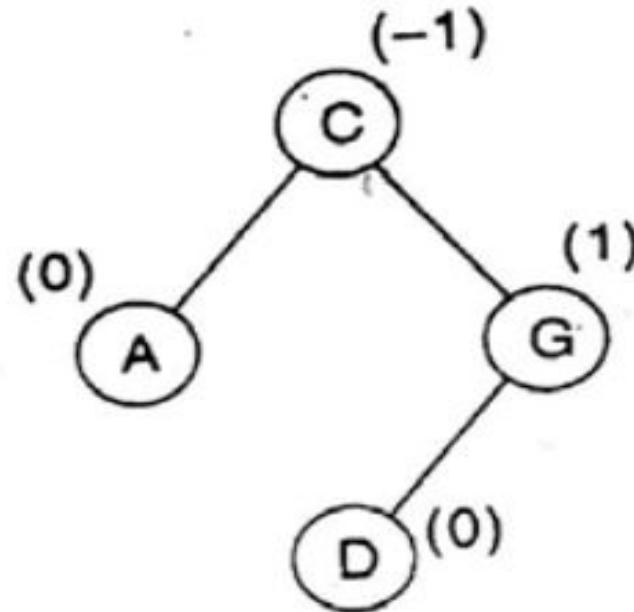


AVL Search Tree [1]

- One of the more popular balanced trees was introduced in 1962 by Adelson-Velskii and Landis and was known as AVL trees.
- **Definition:** An empty binary tree is an AVL tree. A non empty binary tree T is an AVL tree if given T^L and T^R to be the left and right subtrees of T and $h(T^L)$ and $h(T^R)$ to be the heights of subtrees T^L and T^R respectively, T^L and T^R are AVL trees and
$$|h(T^L) - h(T^R)| \leq 1$$
- $h(T^L) - h(T^R)$ is known as the **balance factor (BF)** and for an AVL tree the balance factor of a node can be either 0, 1, or -1.

AVL Search Tree [1]

- Representation of an AVL search Tree:





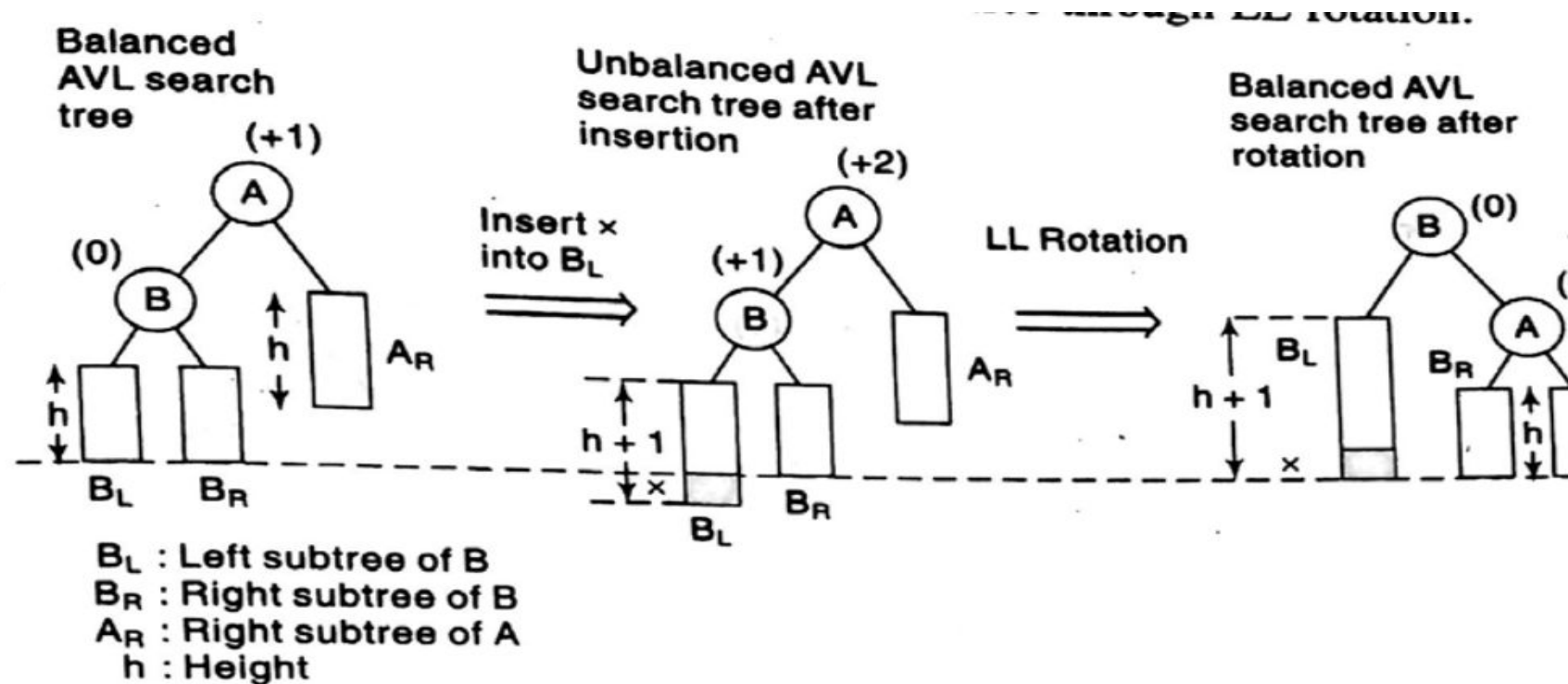
AVL Search Tree [1]

- **Insertion in an AVL search Tree:**

- Insertion an element into an AVL search tree in its first phase is similar to that of the one used in a binary search tree.
- However, if after insertion of the element, the balance factor of any node in the tree is affected so as to render the binary search tree unbalanced, we resort to techniques called **Rotations**.
- **LL Rotation:** Insertion node is in the left subtree of left subtree of node A.
- **RR Rotation:** Insertion node is in the right subtree of right subtree of node A.
- **LR Rotation:** Insertion node is in the right subtree of left subtree of node A.
- **RL Rotation** Insertion node is in the left subtree of right subtree of node A.

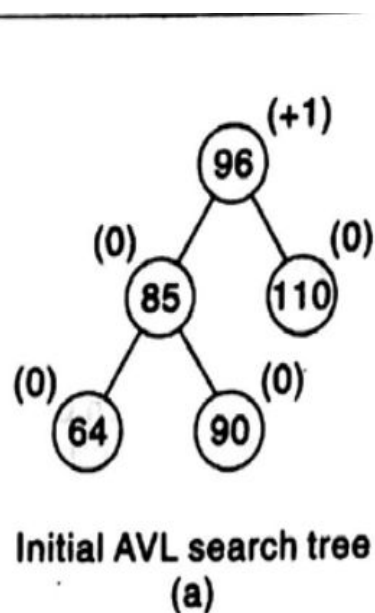
AVL Search Tree [1]

- **LL Rotation:** Insertion node is in the left subtree of left subtree of node A.

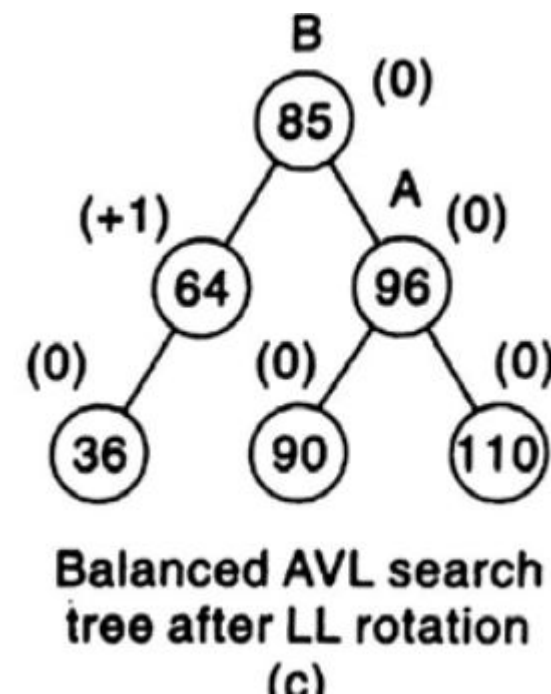
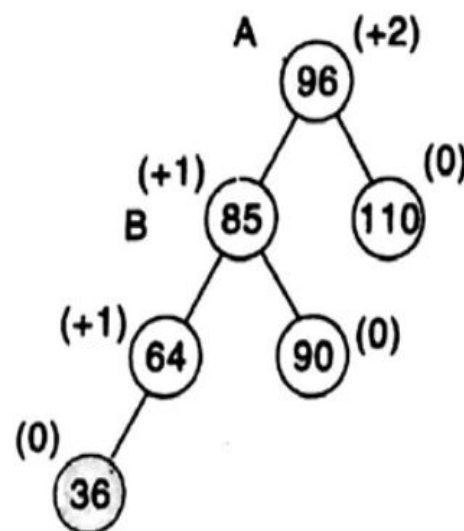


AVL Search Tree [1]

- **LL Rotation:** Insertion node is in the left subtree of left subtree of node A.

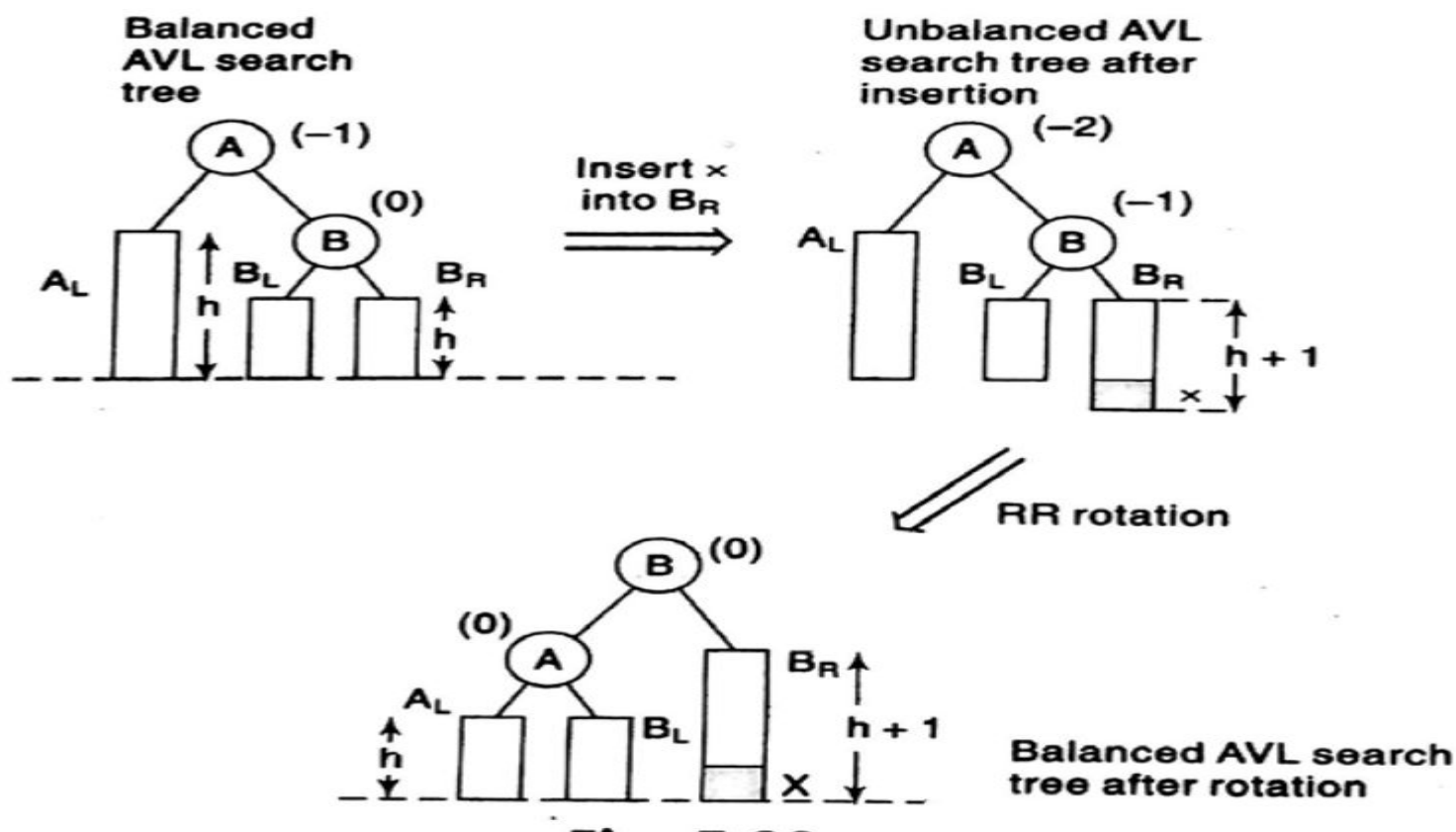


Insert 36
⇒



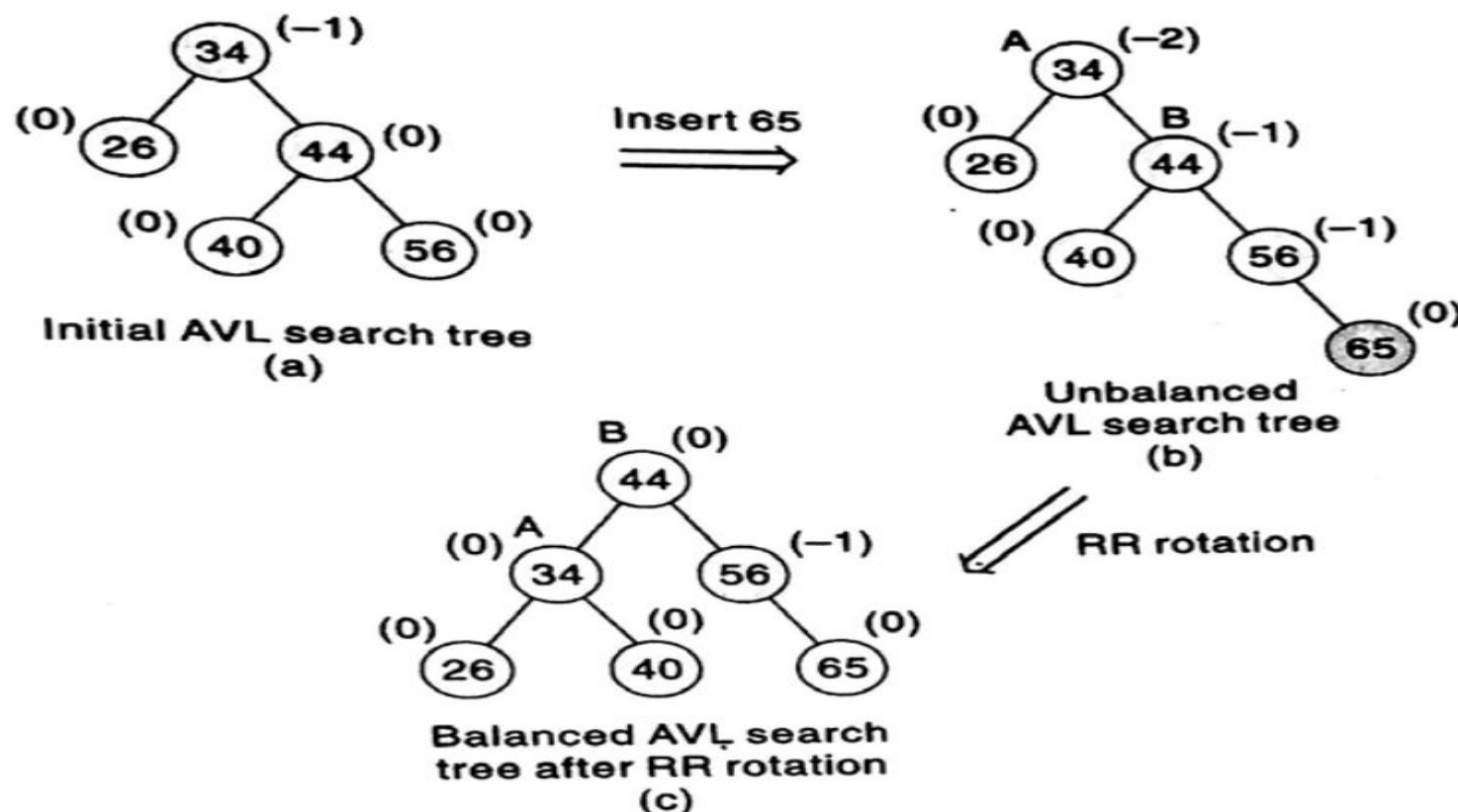
AVL Search Tree [1]

- **RR Rotation:** Insertion node is in the right subtree of right subtree of node A.



AVL Search Tree [1]

- **RR Rotation:** Insertion node is in the right subtree of right subtree of node A.

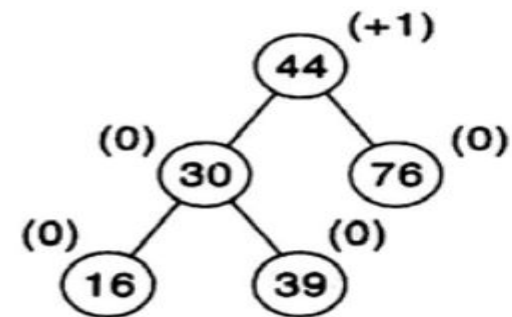


AVL Search Tree: LR Rotation[1]



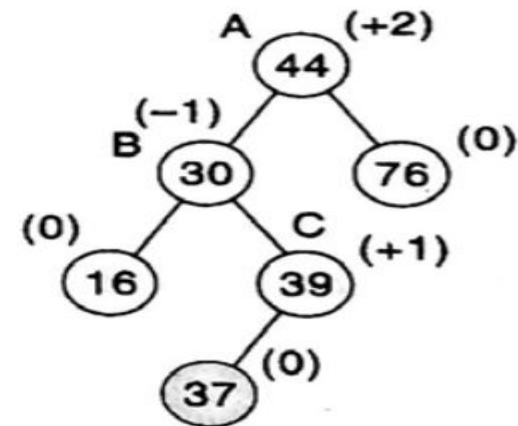
- In this case the Balance Factor values of nodes A and B after balancing are dependent on the BF value of node C after insertion:
 - If $BF(C)=0$ after insertion then $BF(A)=BF(B)=0$, after rotation.
 - If $BF(C)=-1$ after insertion then $BF(A)=0$, $BF(B)=1$, after rotation.
 - If $BF(C)=1$ after insertion then $BF(A)=-1$, $BF(B)=0$, after rotation

AVL Search Tree: LR Rotation[1]

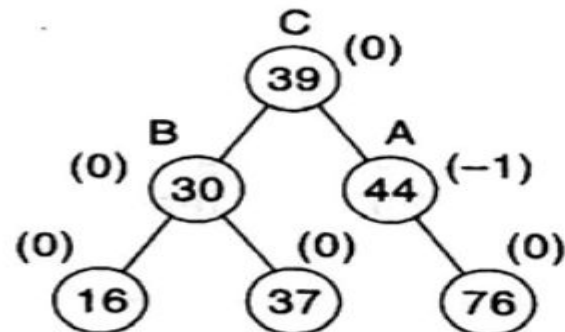


Initial AVL search tree (a)

Insert 37
⇒

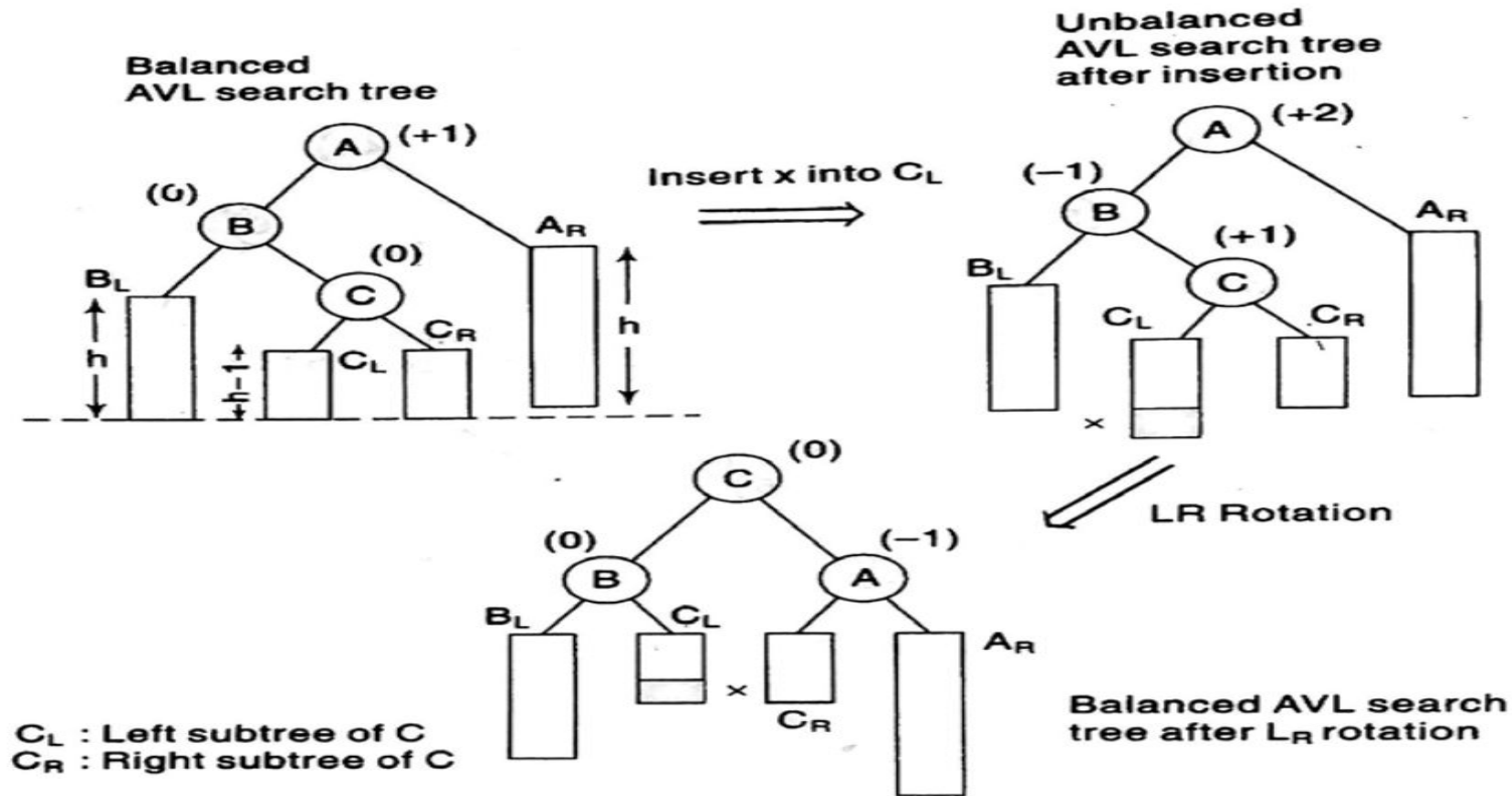


Unbalanced AVL search tree (b)



LR rotation

AVL Search Tree: LR Rotation [1]



AVL Tree [1]



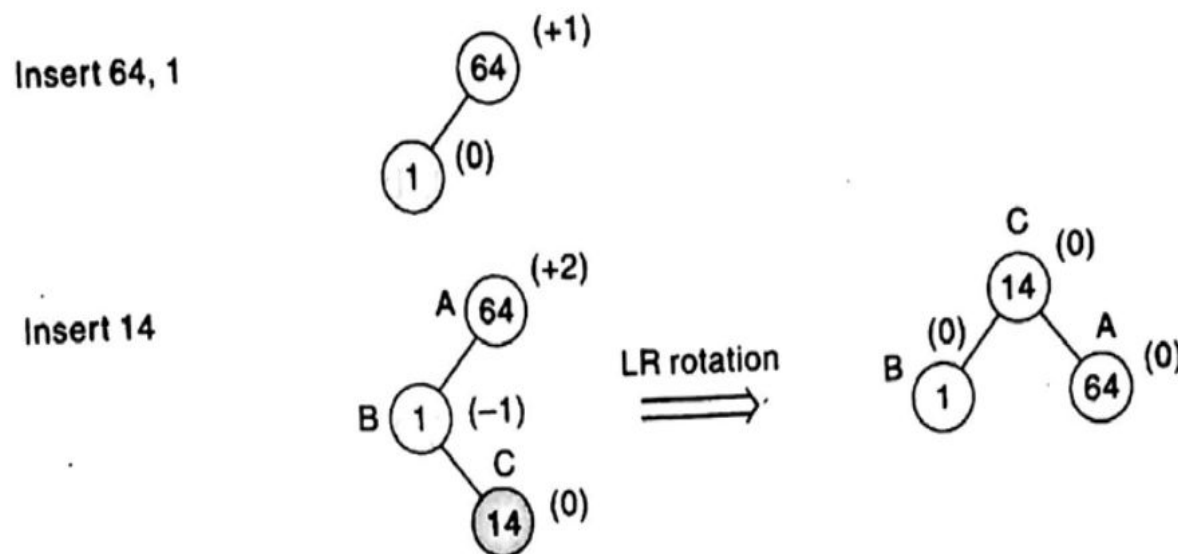
- **Problem:** Construct the AVL tree by inserting following Elements in the order of their occurrence:

64, 1, 14, 26, 13, 110, 98, 85

AVL Tree [1]

- **Problem:** Construct the AVL tree by inserting following Elements in the order of their occurrence:

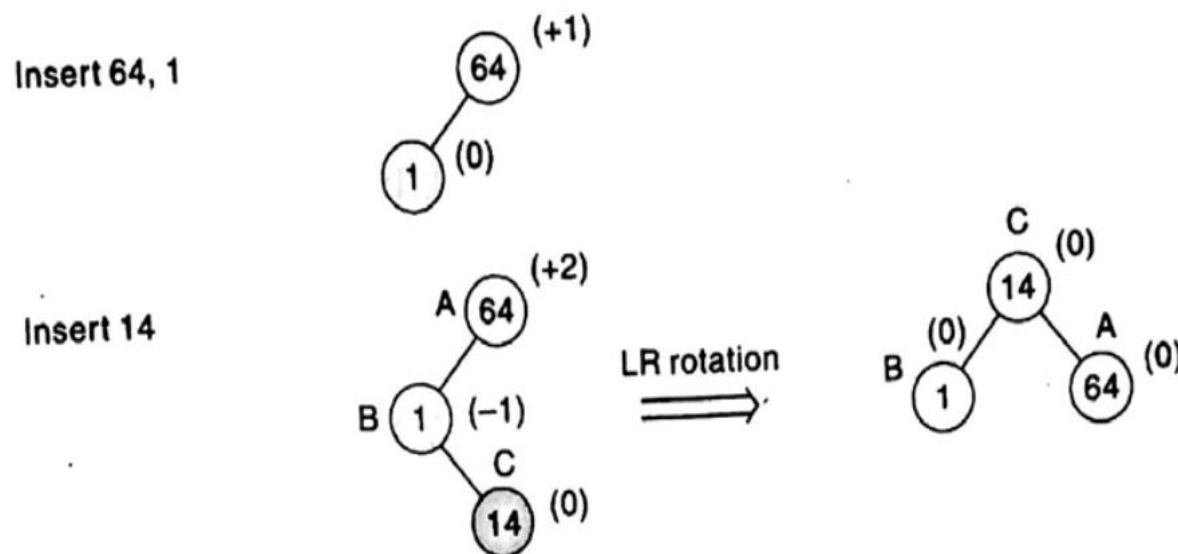
64, 1, 14, 26, 13, 110, 98, 85



AVL Tree [1]

- **Problem:** Construct the AVL tree by inserting following Elements in the order of their occurrence:

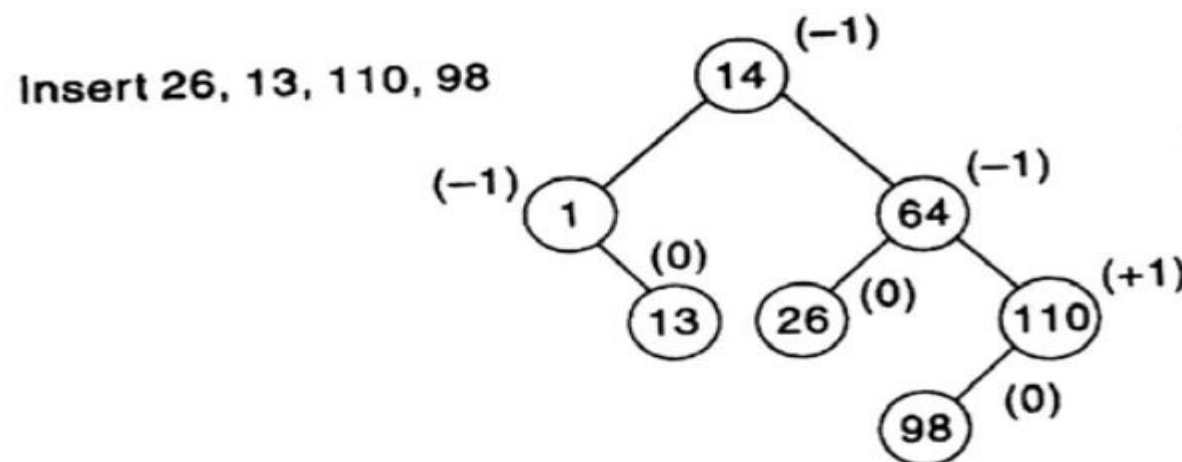
64, 1, 14, 26, 13, 110, 98, 85



AVL Tree [1]

- **Problem:** Construct the AVL tree by inserting following Elements in the order of their occurrence:

64, 1, 14, 26, 13, 110, 98, 85

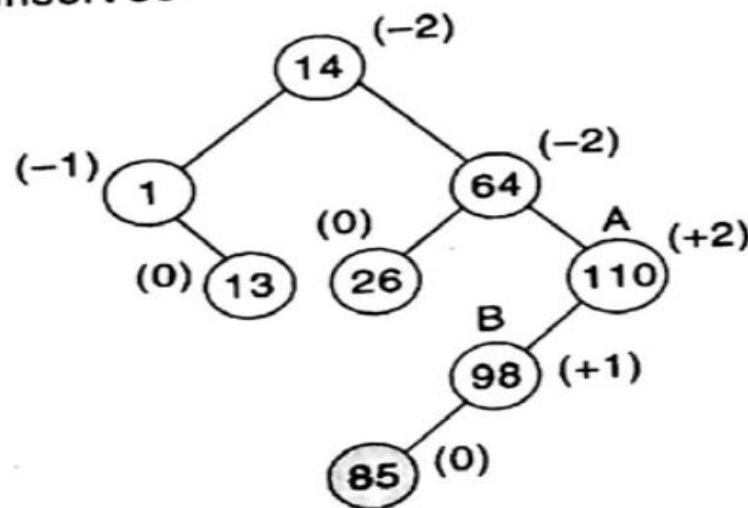


AVL Tree [1]

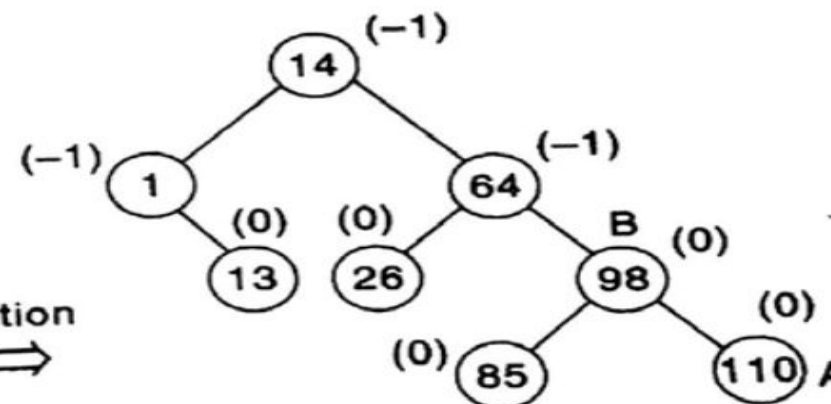
- **Problem:** Construct the AVL tree by inserting following Elements in the order of their occurrence:

64, 1, 14, 26, 13, 110, 98, 85

Insert 85



LL rotation
⇒





AVL Search Tree [1]

- **Deletion in an AVL search Tree:**

- On deletion of a node X from the AVL tree, let A be the closest ancestor node on the path from X to the root side, with a balance factor of +2 or -2.
- To restore balance the rotation is first classified as L or R depending on whether the deletion occurred on the left or right subtree of A.
- Now depending on the value of $BF(B)$ where B is the root of the left or right subtree of A, the R or L imbalance is further classified as R0, R1 and R-1 or L0, L1 and L-1.

- **R0 Rotation:**

- **R1 Rotation:**

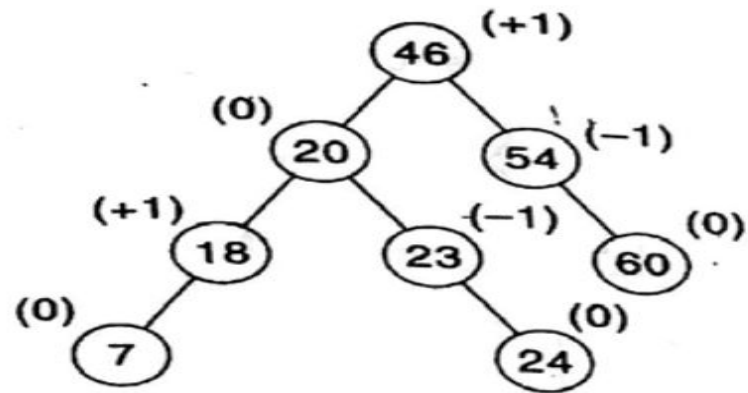
- **R -1 Rotation:**

- **L0Rotation:**

- **L1Rotation:**

- **L-1Rotation:**

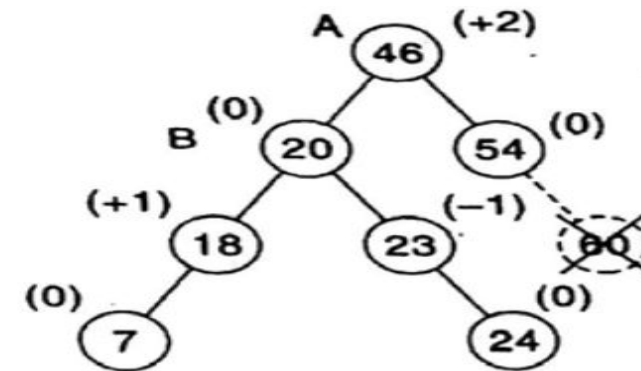
AVL Search Tree: R0 Rotation [1]



AVL search tree
before deletion

(a)

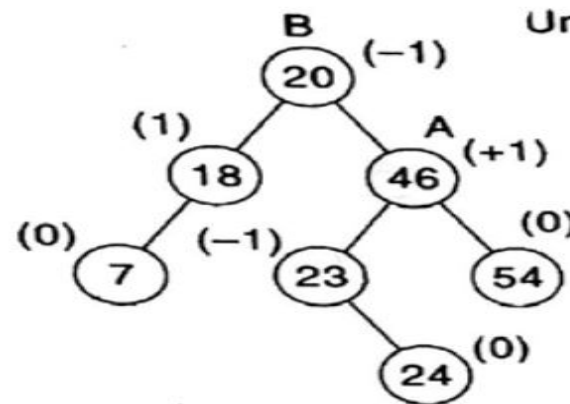
Delete 60
 $\Rightarrow$



Unbalanced AVL search tree
after deletion of 60

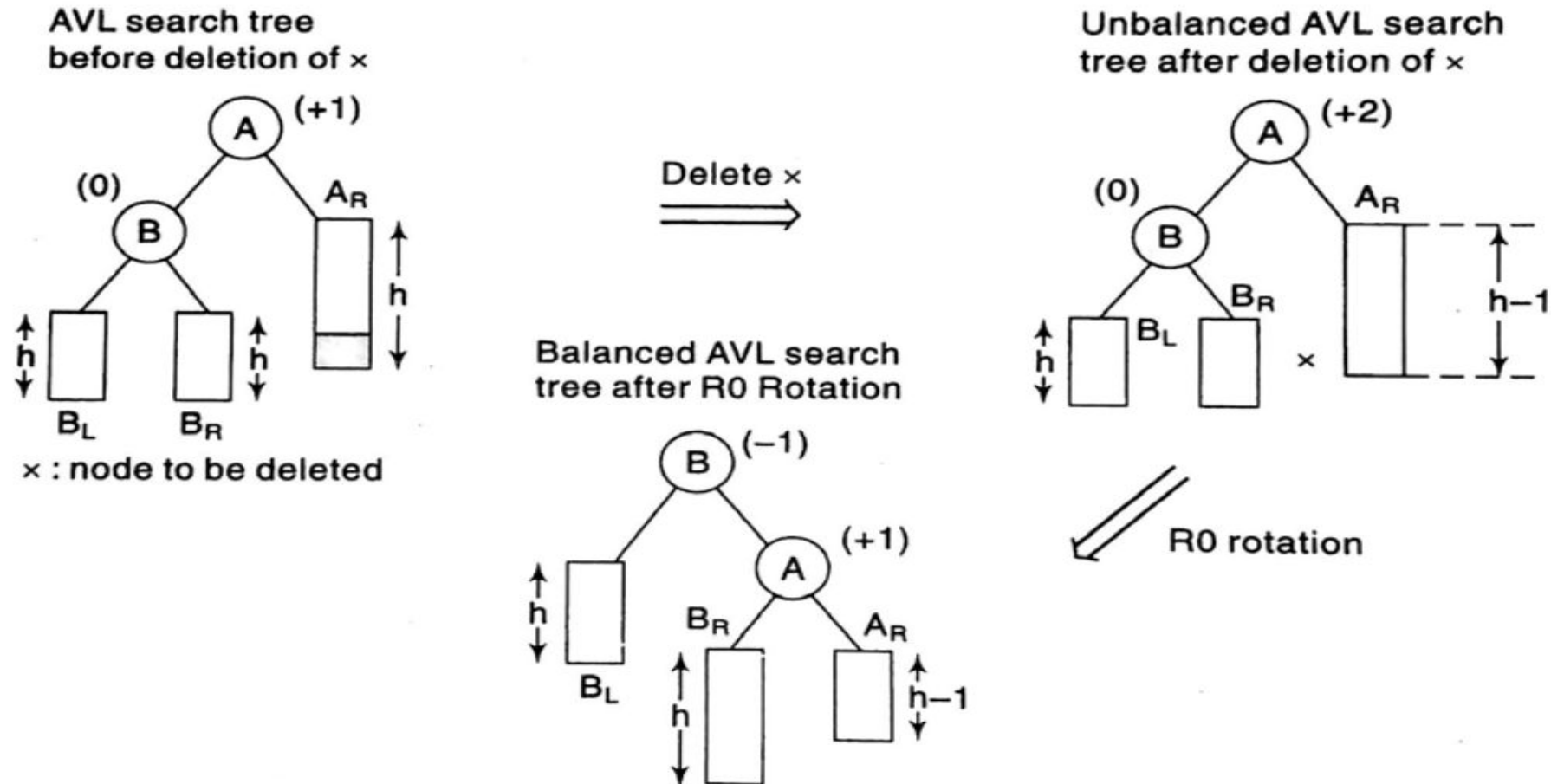
(b)

R0 rotation

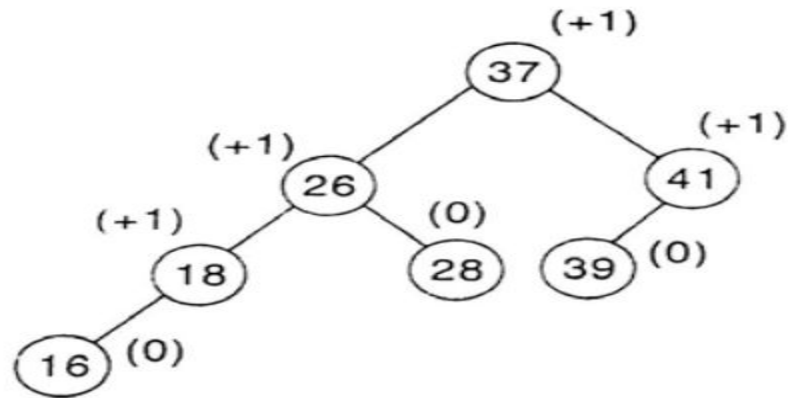


Balanced AVL search tree after R0 rotation

AVL Search Tree: R0 Rotation [1]

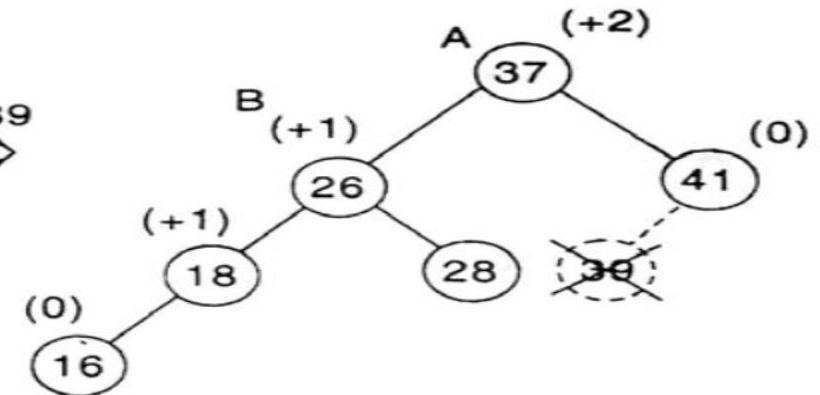


AVL Search Tree: R1 Rotation [1]

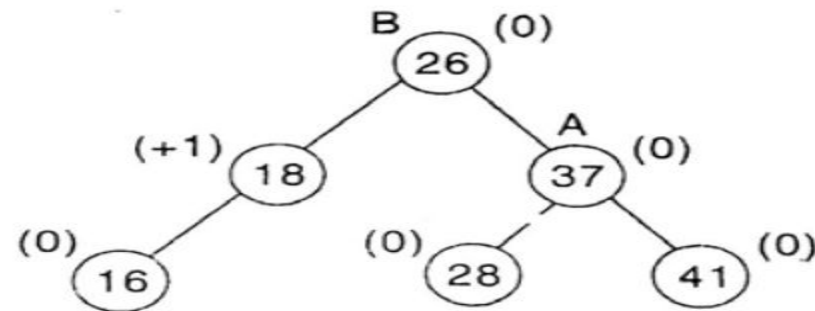


AVL search tree
before deletion
(a)

Delete 39
⇒



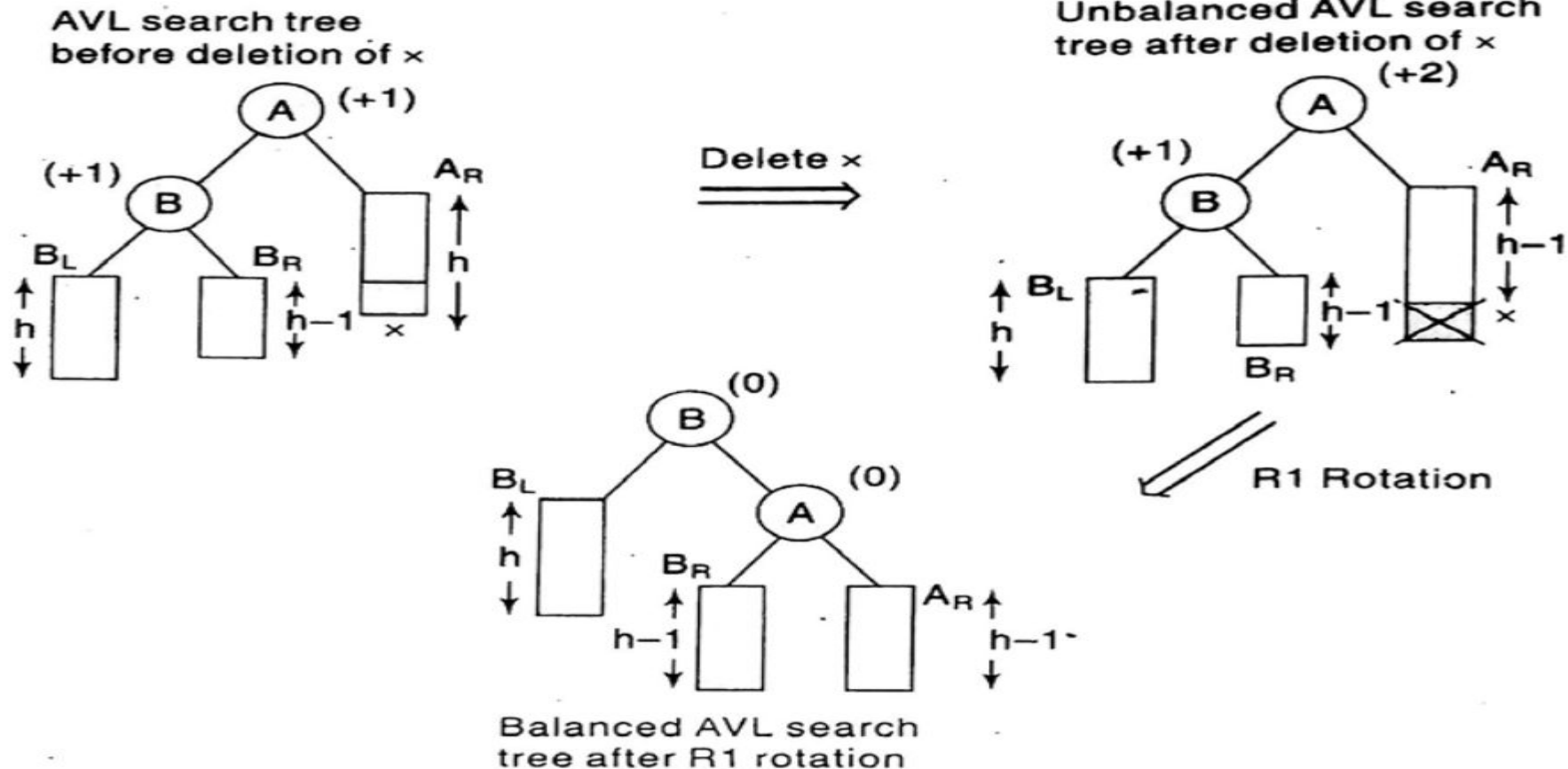
Unbalanced AVL search tree
after deletion of 39
(b)



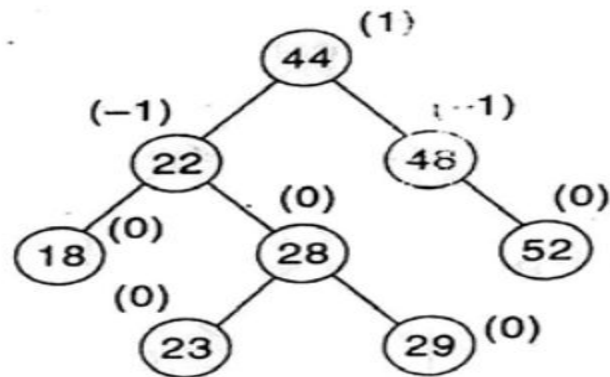
Balanced AVL search tree after R1 rotation
(c)

R1 rotation

AVL Search Tree: R1 Rotation [1]

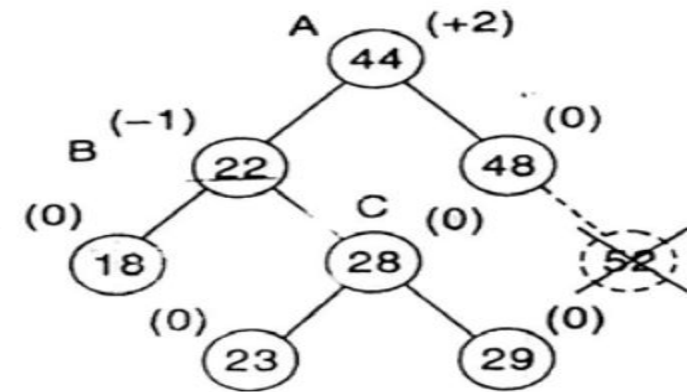


AVL Search Tree: R-1 Rotation [1]

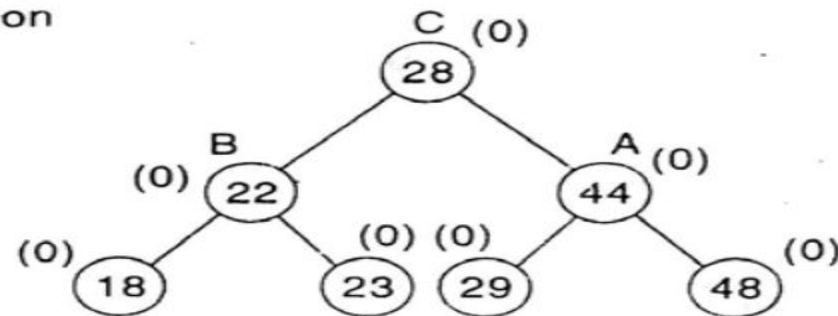


AVL search tree
before deletion
(a)

Delete 52
 $\Rightarrow$



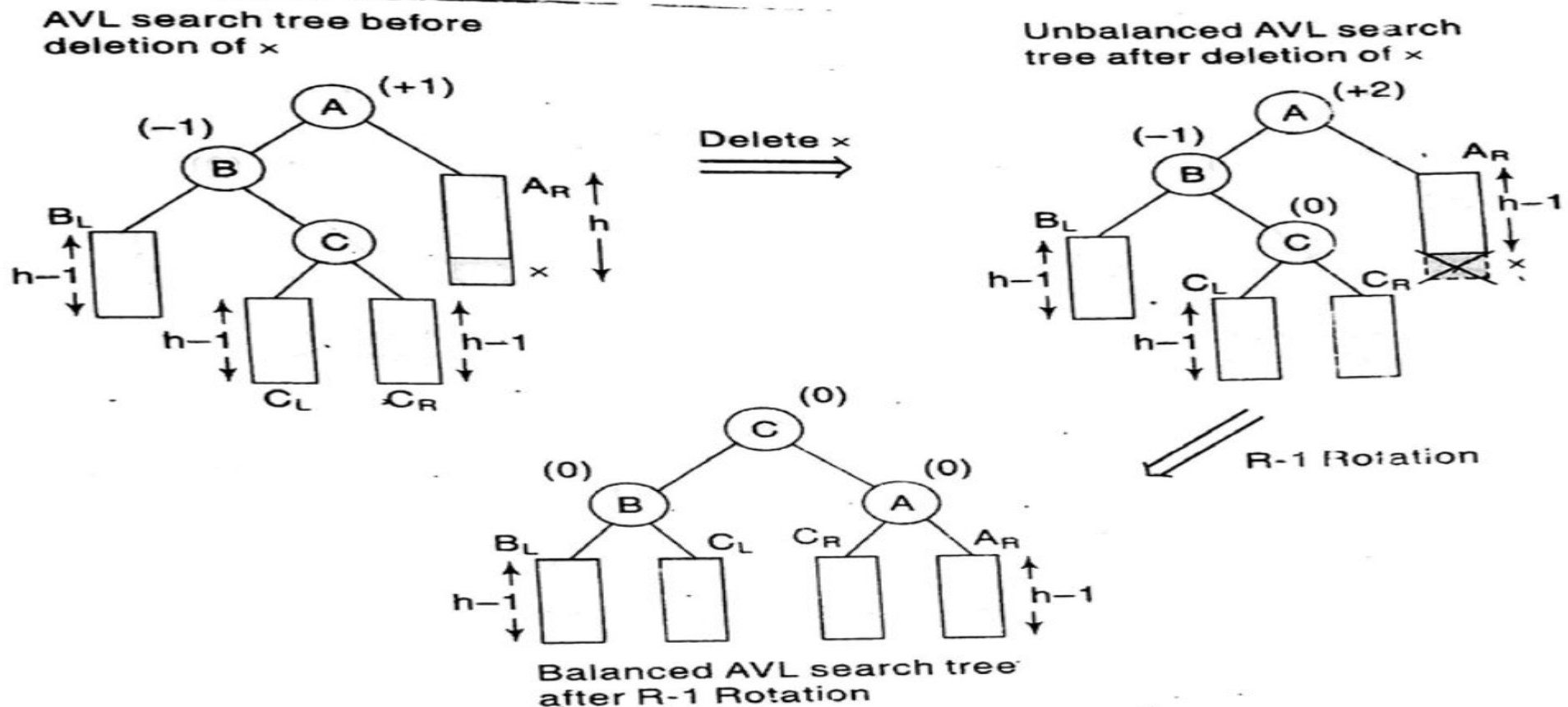
Unbalanced AVL search tree
after deletion
(b)



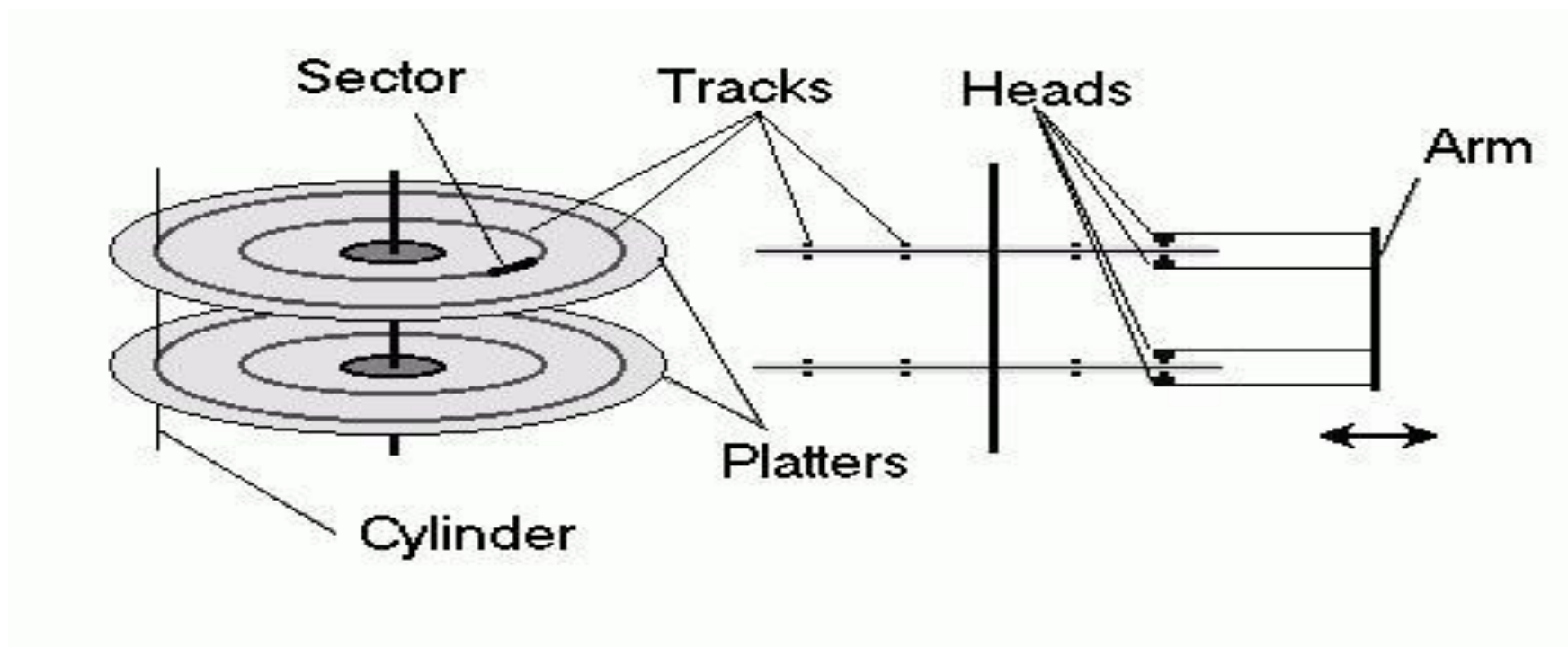
Balanced AVL search tree after R-1 Rotation
(c)

$\swarrow$ R-1 Rotation

AVL Search Tree: R-1 Rotation [1]



Disk Access[3]





M-way Tree [1]

- ❖ All the data structures discussed so far favour data stored in the **internal memory and support internal information**.
- ❖ However, to favour **retrieval and manipulation of data stored in external memory**, viz. storage devices such as **disks** etc.
- ❖ There is need for some special data structure.
- ❖ **M-way** search tree, **B tree** and **B+** tree are examples.



M-way Tree [1]

- ❖ M-way search trees are generalized versions of binary search trees.
- ❖ The goal of m-way search tree is to minimize the accesses while retrieving a key from a file.
- ❖ However, an m-way search tree of height h calls for $O(h)$ number of accesses for an insert/delete/retrieval operation.
- ❖ Hence it pays to ensure that the height h is close to $\log_m(n + 1)$, because the number of elements in an m-way search tree of height h ranges from a minimum of h to a maximum of $m^h - 1$.



M-way Tree [1]

- ❖ This implies that an m -way search tree of n elements ranges from a minimum height of $\log_m(n + 1)$ to a maximum height of n .
- ❖ Therefore there arises the need to maintain balanced m -way search trees.
- ❖ B trees are balanced m -way search trees.



M-way Tree [1]

❖ Definition:

- ❖ An m-way search tree T may be an empty tree. If T is non empty, it satisfies the following properties:
 - ❖ For some **integer m**, known as **order of the tree**, each node is of degree which can reach a maximum of m, in other words, each node has **at most m child nodes**.
 - ❖ A node may be represented as $A_0, (K_1, A_1), (K_2, A_2) \dots (K_{m-1}, A_{m-1})$ where, $K_i, 1 \leq i \leq m-1$ are the keys and $A_i, 0 \leq i \leq m-1$ are pointers to subtrees of T.

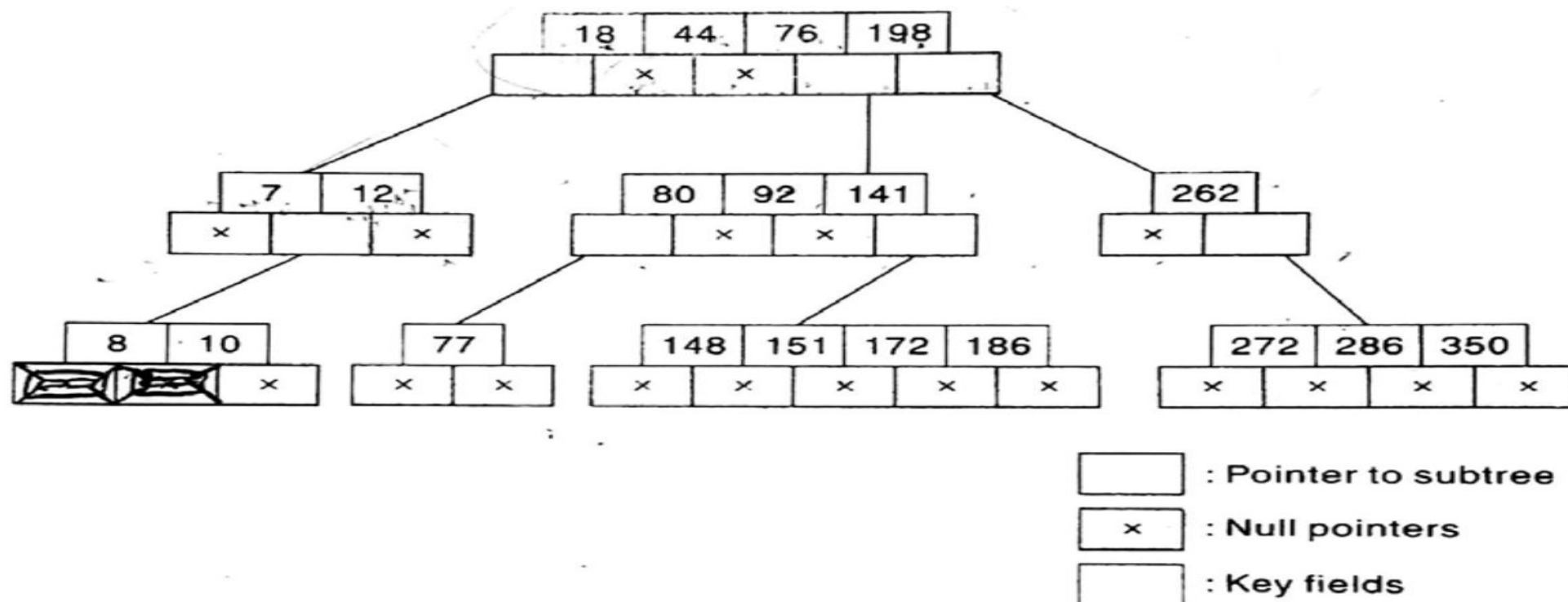


M-way Tree [1]

- ❖ If a node has k child nodes where $k \leq m$, then the node can have only $(k - 1)$ keys $K_1, K_2 \dots K_{k-1}$, contained in the node such that $k_i \leq k_{i+1}$ and each of the keys partitions all the keys in the subtrees into k subsets.
- ❖ For a node $A_0, (K_1, A_1), (K_2, A_2) \dots (K_{m-1}, A_{m-1})$, all key values in the subtree pointed to by A_i are less than the key K_{i+1} , $0 \leq i \leq m - 2$, and all key values in the subtree pointed to by A_{m-1} are greater than K_{m-1} .
- ❖ Each of the subtrees $A_i, 0 \leq i \leq m - 1$ are also m -way search trees.

M-way Tree [1]

❖ Example 5-way tree:



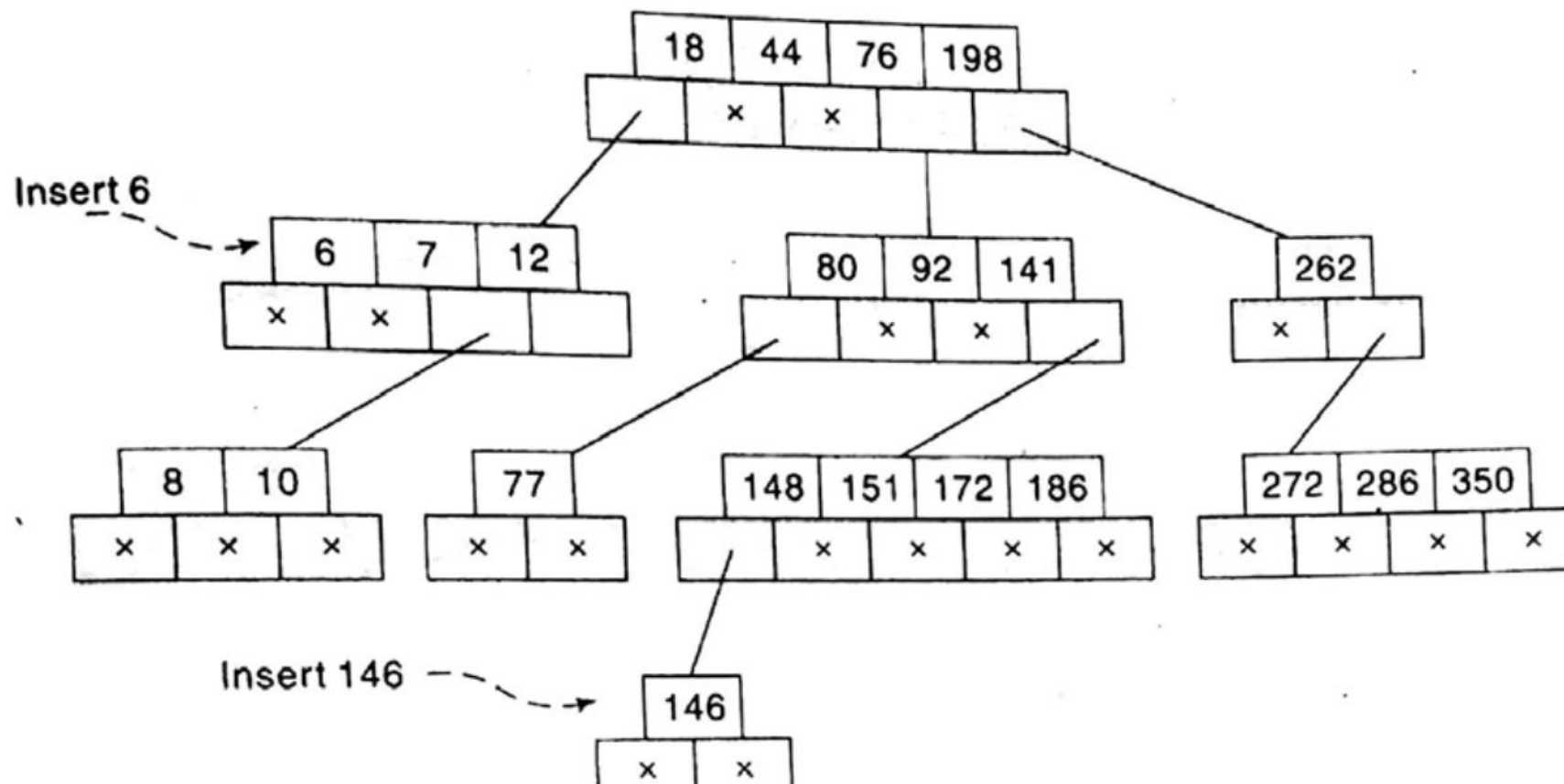


M-way Tree [1]

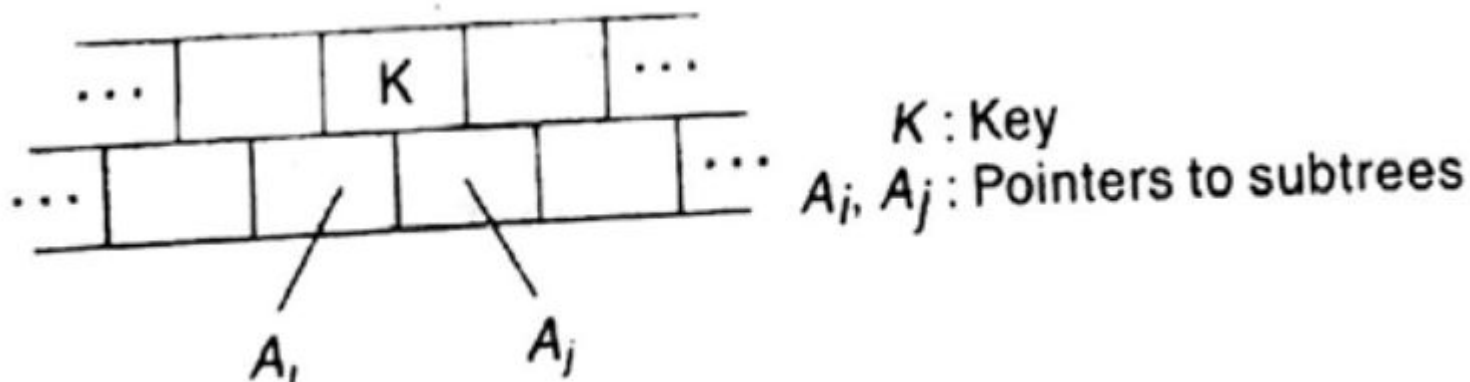
❖ Operations on M-way Search Tree:

- ❖ **Searching:** Similar to the Binary Search tree.
- ❖ **Insertion:**
- ❖ **Deletion:**

Insertion in M-way Tree [1]



Deletion in M-way Tree [1]



- ❖ Let K be a key to be deleted from the m -way search tree. To delete the key we proceed as one would to search for the key.



Deletion in M-way Tree [1]

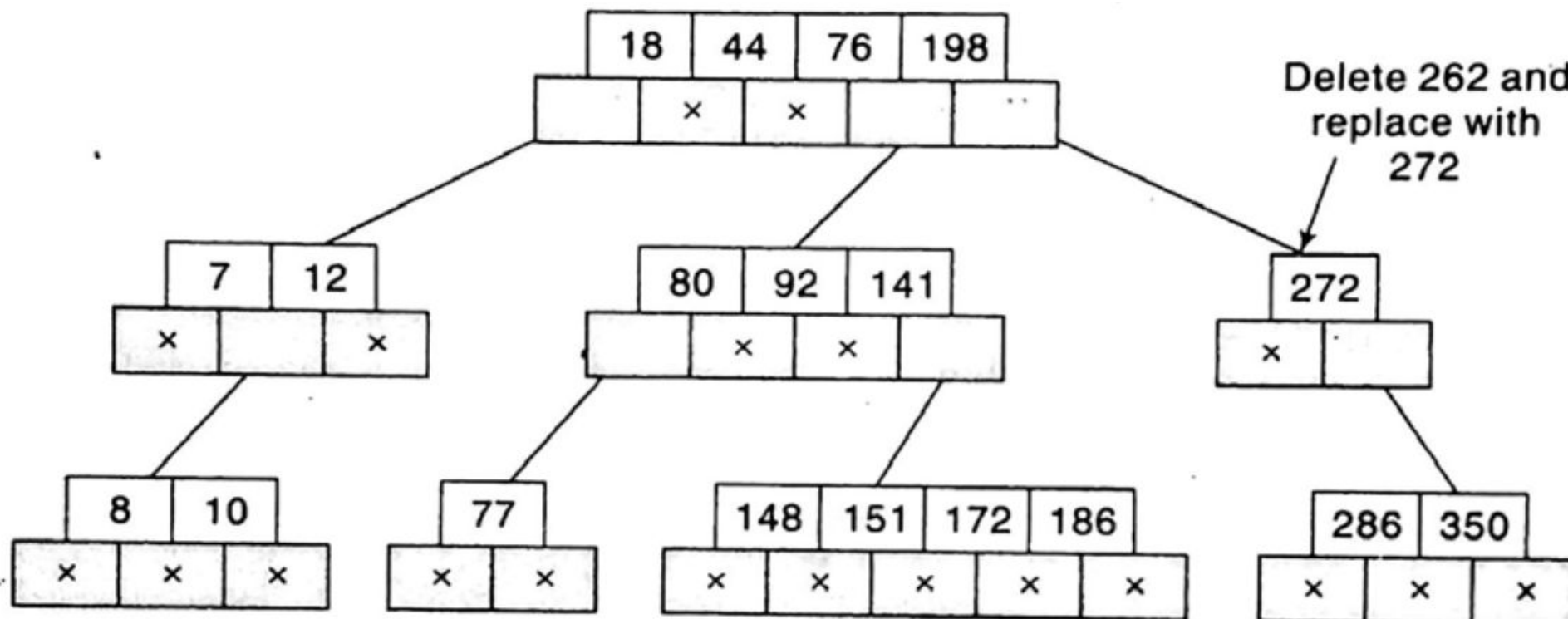
- ❖ If $(A_i = A_j == \text{null})$ then delete K.
- ❖ If $(A_i \neq \text{null}, A_j = \text{null},)$ then choose the largest of the key elements k' in the child node pointed to by A_i , delete the key k' and replace k by k' . The deletion of k' may call for subsequent replacements and therefore in a similar manner, to enable the key k' move up the tree.
- ❖ If $(A_i = \text{null}, A_j \neq \text{null},)$ then choose the smallest of the key elements k'' in the child node pointed to by A_j , delete the key k'' and replace k by k'' . The deletion of k'' may call for subsequent replacements and therefore in a similar manner, to enable the key k'' move up the tree.



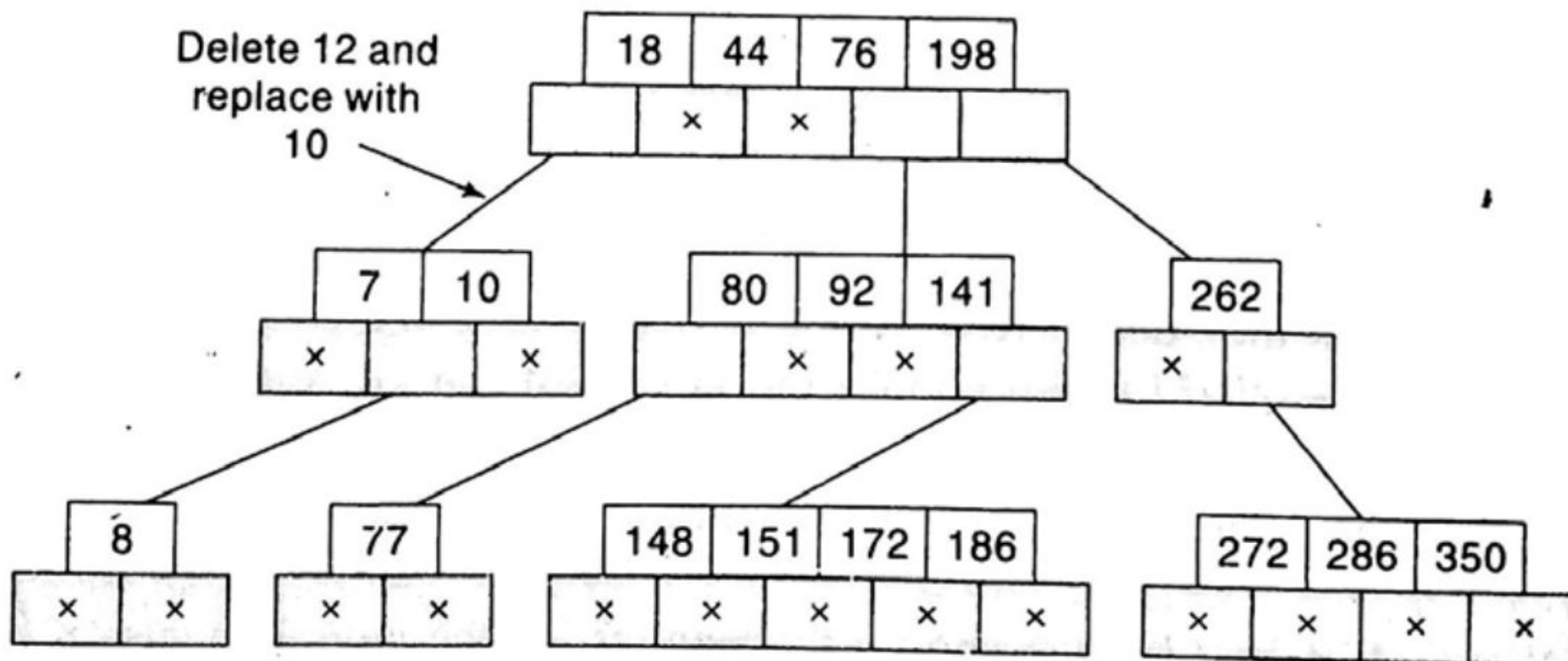
Deletion in M-way Tree [1]

- ❖ If $(A_i \neq \text{null}, A_j \neq \text{null},)$ then choose either the largest of the key elements k' in the child node pointed to by A_i , or the smallest of the key elements k'' from the subtree A_j to replace K. Move k' or k'' up the tree and it may call subsequent replacements and deletions.

Deletion in M-way Tree [1]

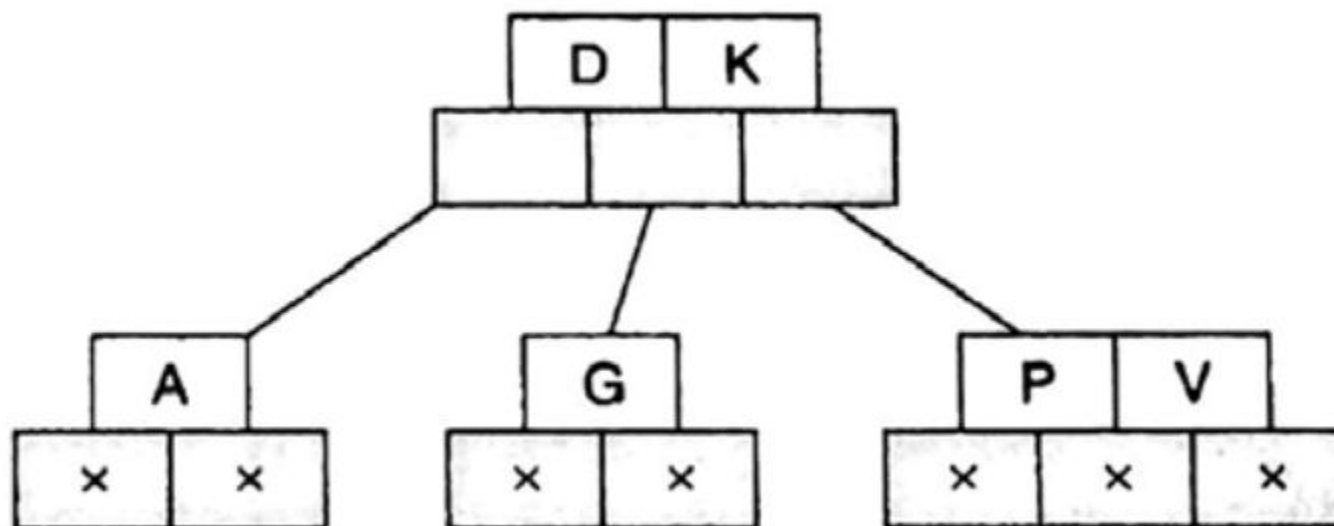


Deletion in M-way Tree [1]



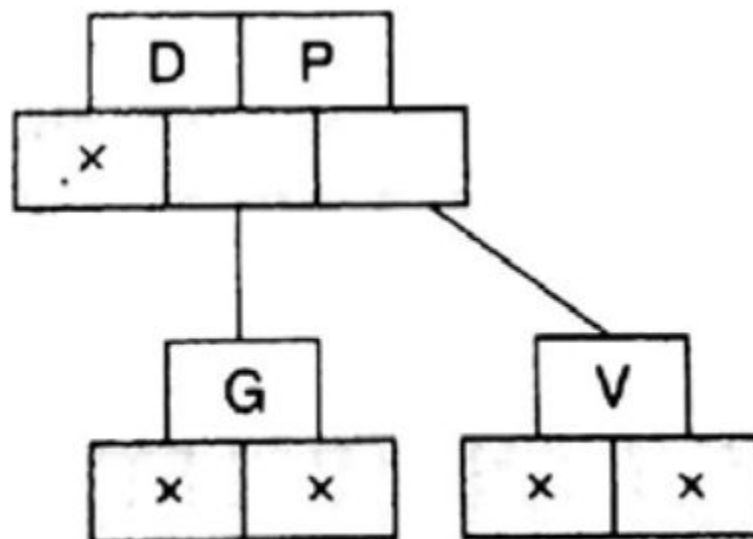
M-way Tree [1]

- ❖ Example: Insert following key using 3-way tree
 - ❖ **D, K, P, V, A, G**



M-way Tree [1]

❖ Example: **Delete A and K**



B Trees [1]



- ❖ M-way search trees have the advantage of minimizing file accesses due to their restricted height.
- ❖ However, it is essential that the height of the tree be kept as low as possible and therefore there arises the need to maintain balanced m-way search trees.

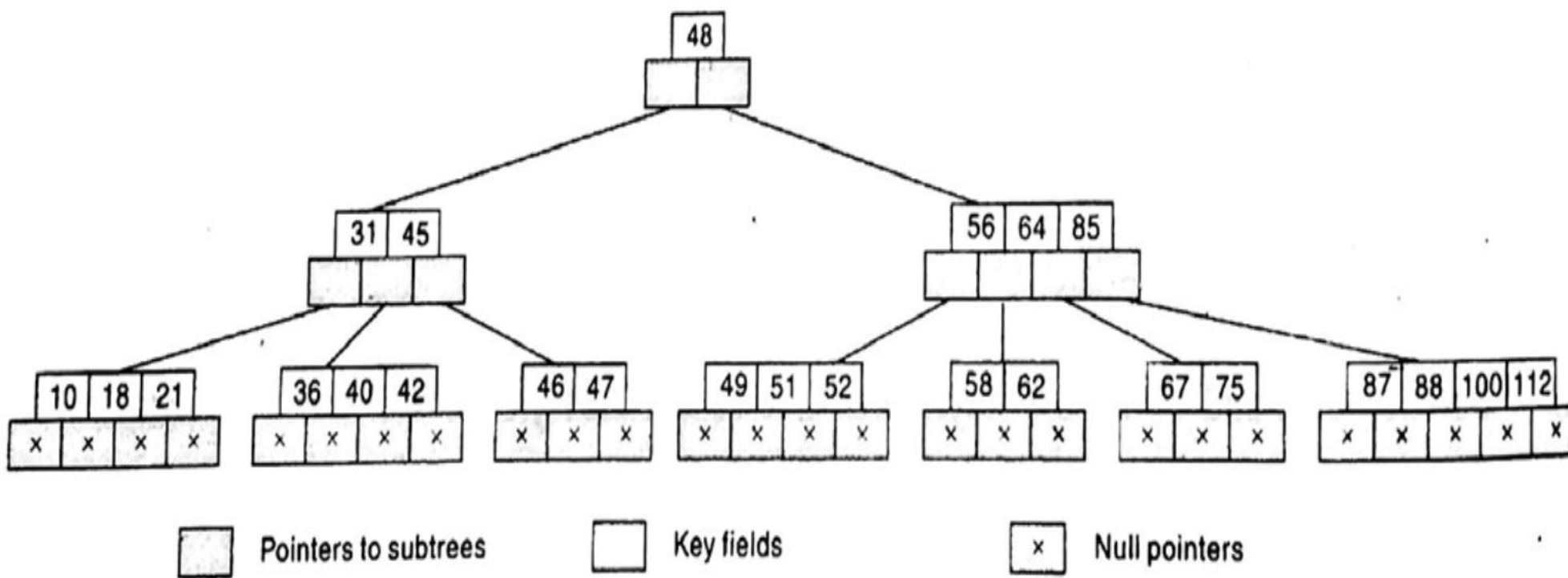


B Trees Definition [1]

- ❖ A B tree of order m , if non empty, is an m -way search tree in which:
 - ❖ The root has as **at least two child nodes** and at **most m child nodes**.
 - ❖ The **internal nodes** except the root have at **least $\lceil \frac{m}{2} \rceil$ child nodes** and at **most m child nodes**.
 - ❖ The number of keys in **each internal node is one less than the number of child nodes** and these keys partition the keys in the subtree of the node in a manner similar to that of m -way search trees.
 - ❖ **All leaf nodes are on the same level.**

B Trees [1]

◆ Example:



B Trees [1]



- ◆ **Operations on B Trees:**
 - ◆ **Searching:** similar to the m-way search trees
 - ◆ **Insertion:**
 - ◆ **Deletion:**



Insertion in B Trees [1]

- ❖ If the leaf node in which the key is to be inserted is not full. Then the insertion is done in the node. A node is said to be full if it contains a maximum of $(m-1)$ keys, given the order of the B-tree to be m .
- ❖ If the node were to be full, then insert the key in order into the existing set of keys in the node split the node at its median into two nodes at the same level, pushing the median element up by one level.

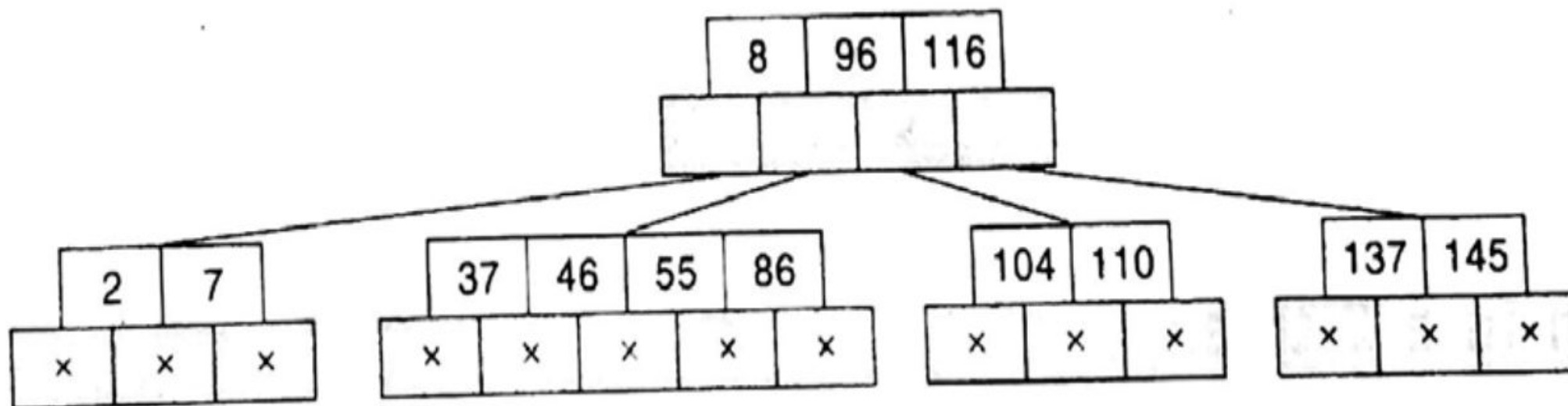
Insertion in B Trees [1]



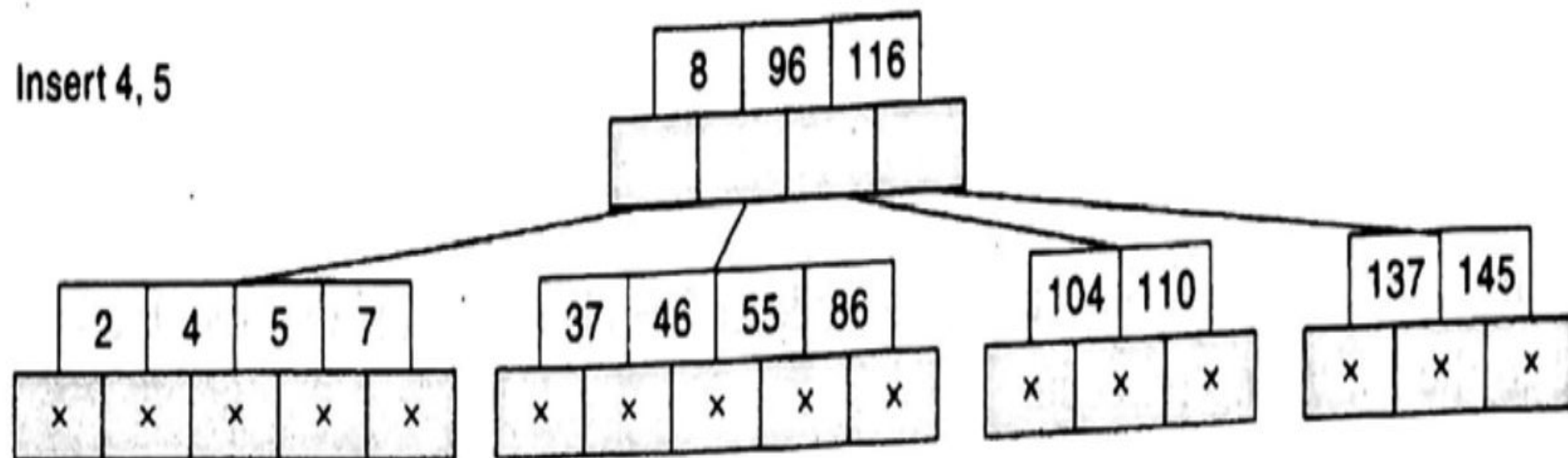
- ❖ Note that the split nodes are only half full. Accommodate the median element in the parent node if it is not full.
- ❖ Otherwise repeat the same procedure and this may even call for rearrangement of keys in the root node or the formation of a new root itself.

Insertion in B Trees [1]

◆ **Example:** Given B tree of order 5. Insert 4, 5, 58, 6.



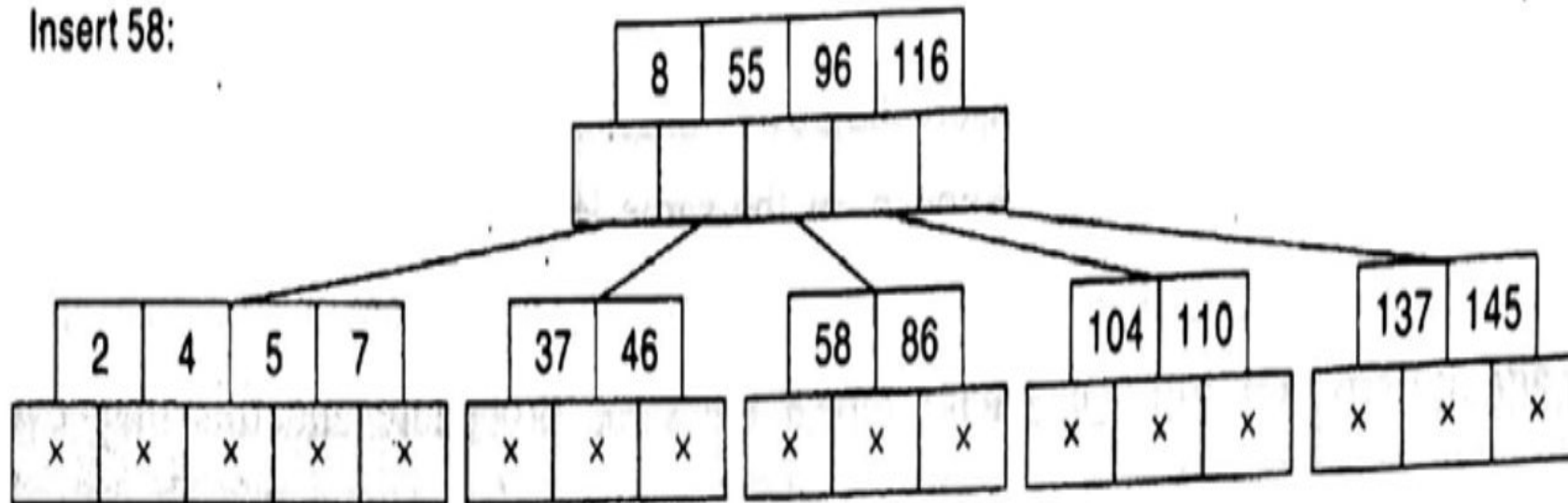
Insertion in B Trees [1]



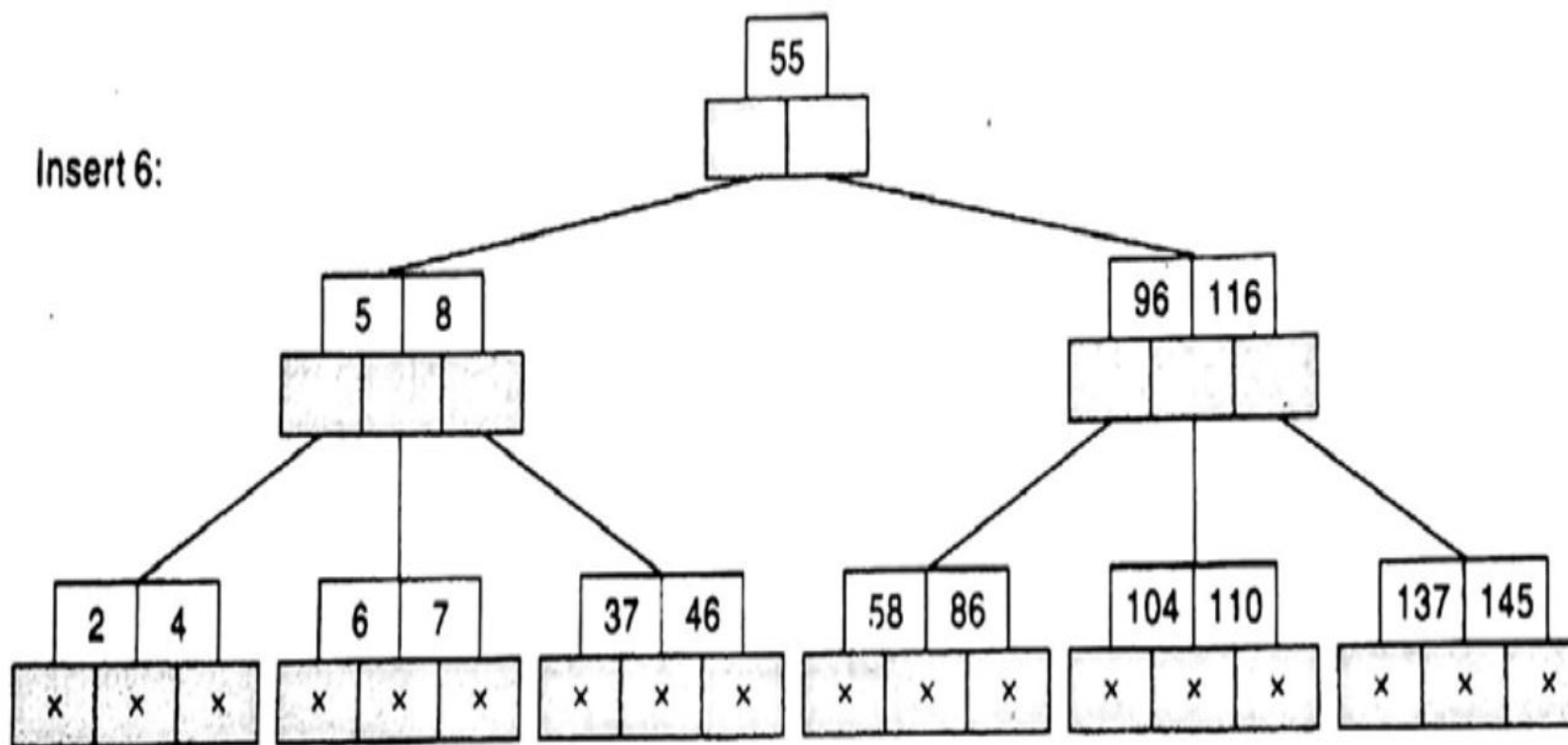
Insertion in B Trees [1]



Insert 58:

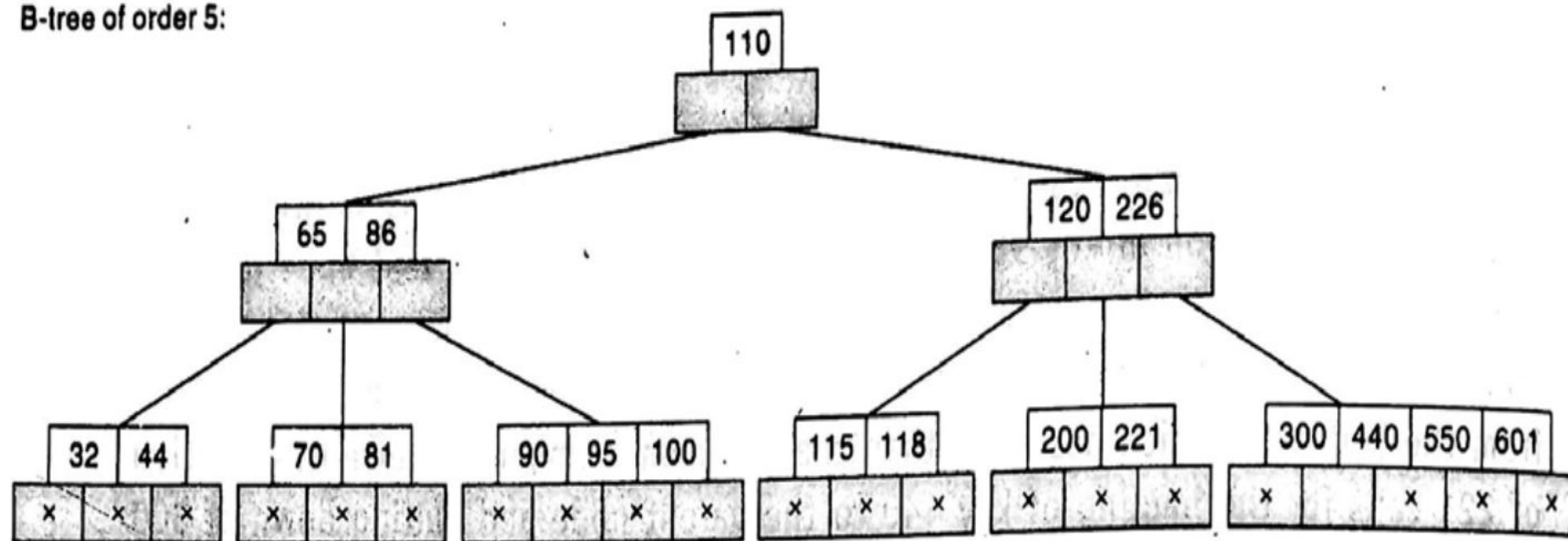


Insertion in B Trees [1]



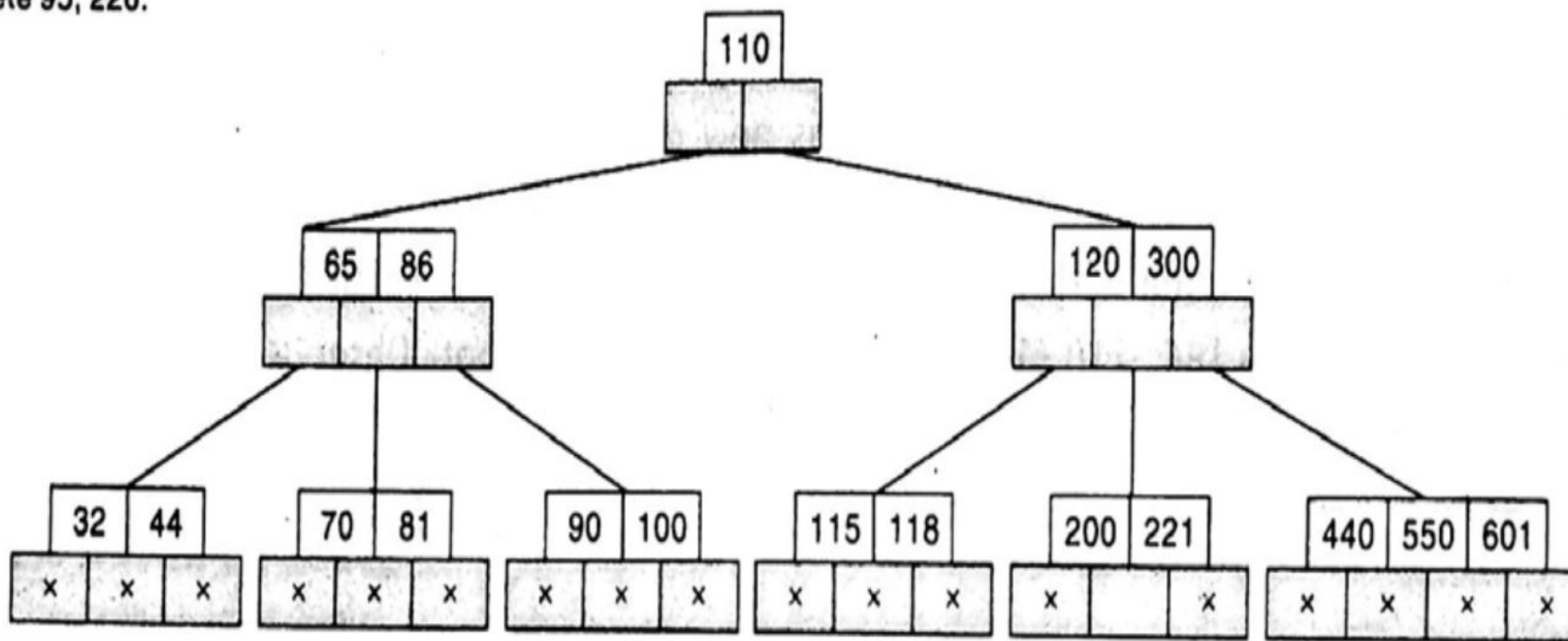
Deletion in B Trees [1]

B-tree of order 5:



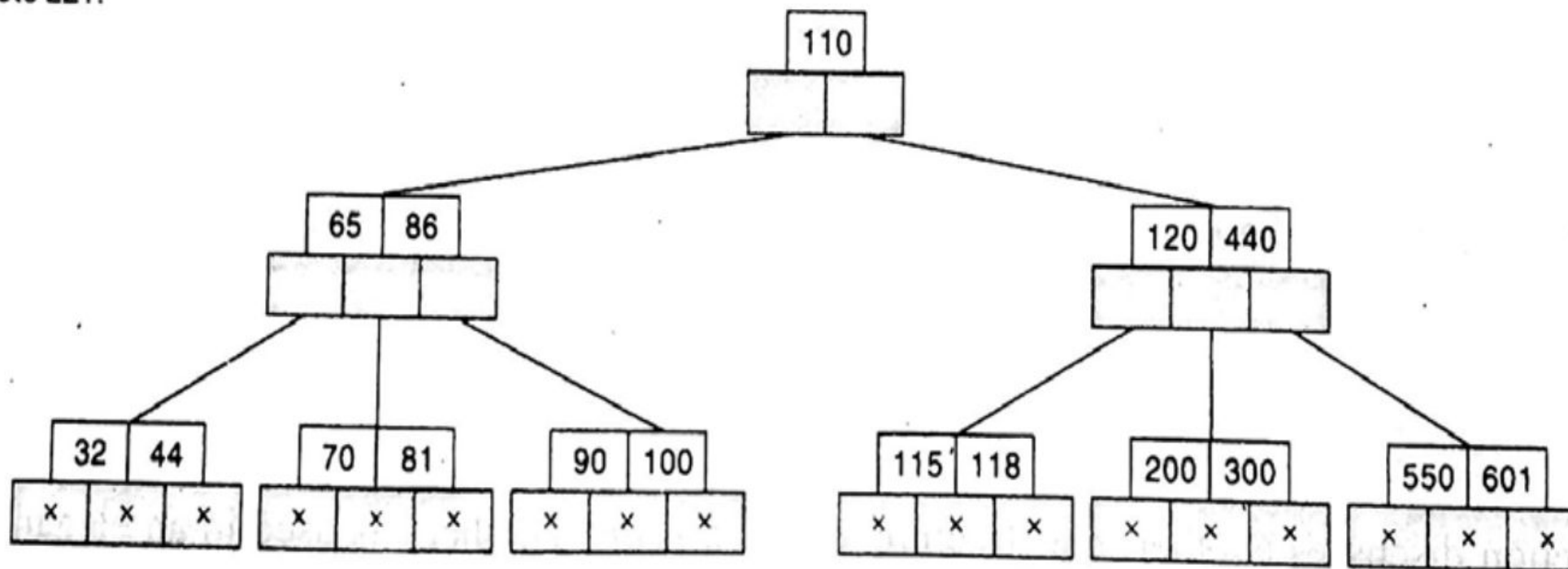
Deletion in B Trees [1]

Delete 95, 226:



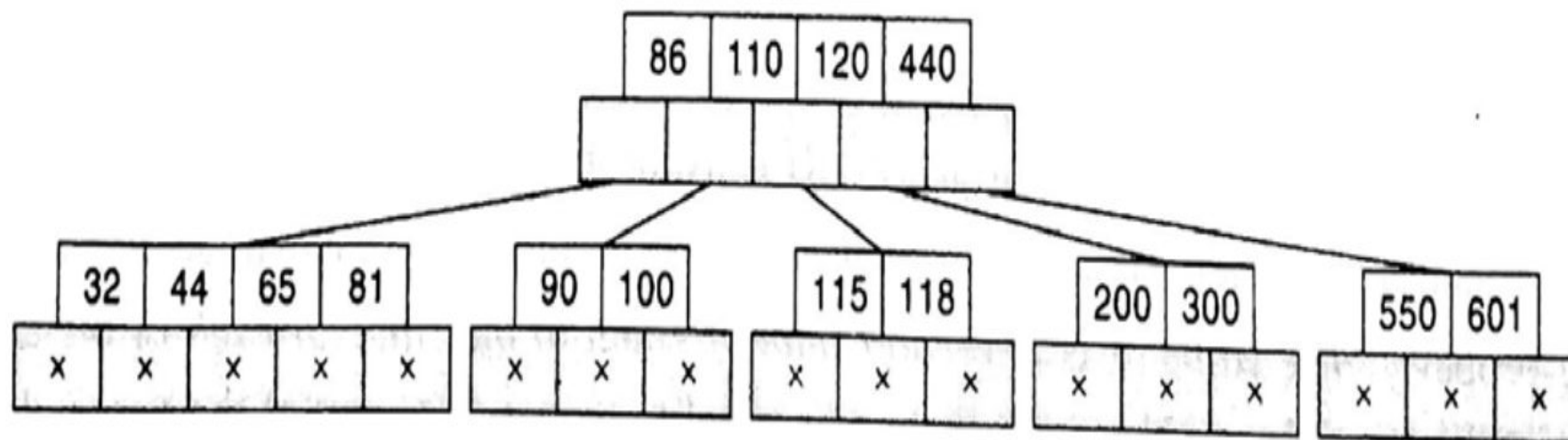
Deletion in B Trees [1]

Delete 221:



Deletion in B Trees [1]

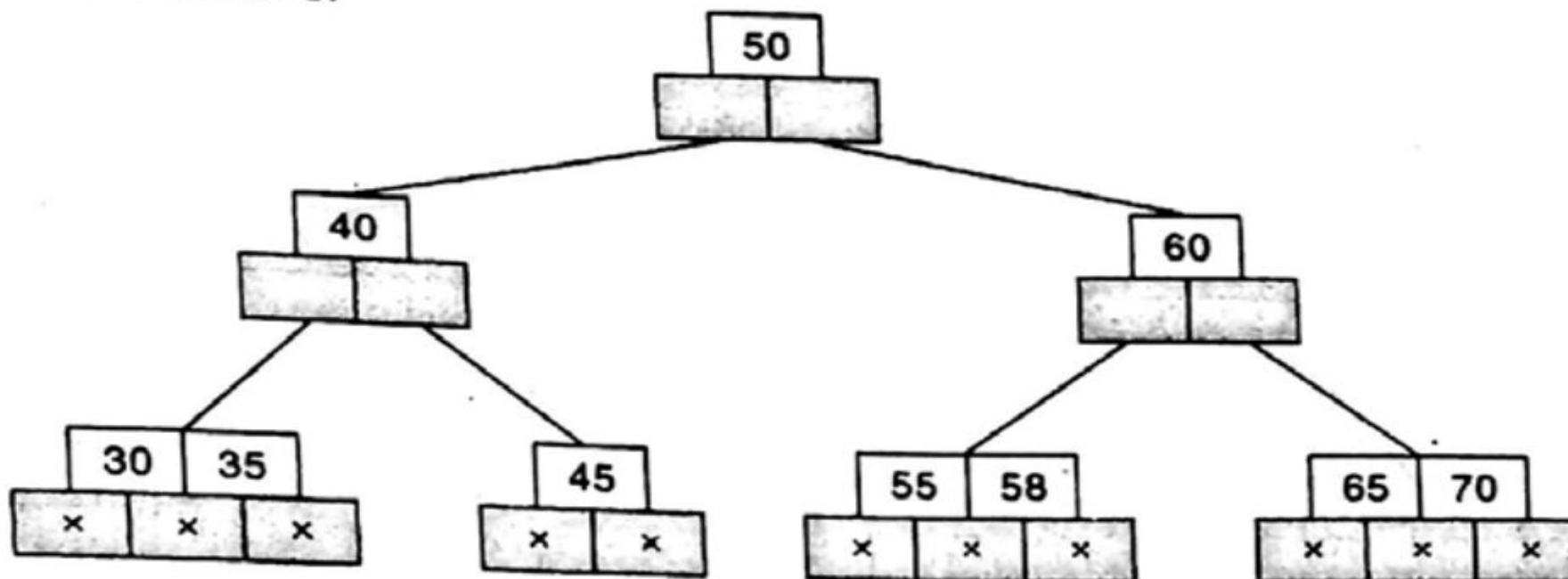
Delete 70:



Deletion in B Trees [1]

Example: Insert 75, 57, deletion 35, 65

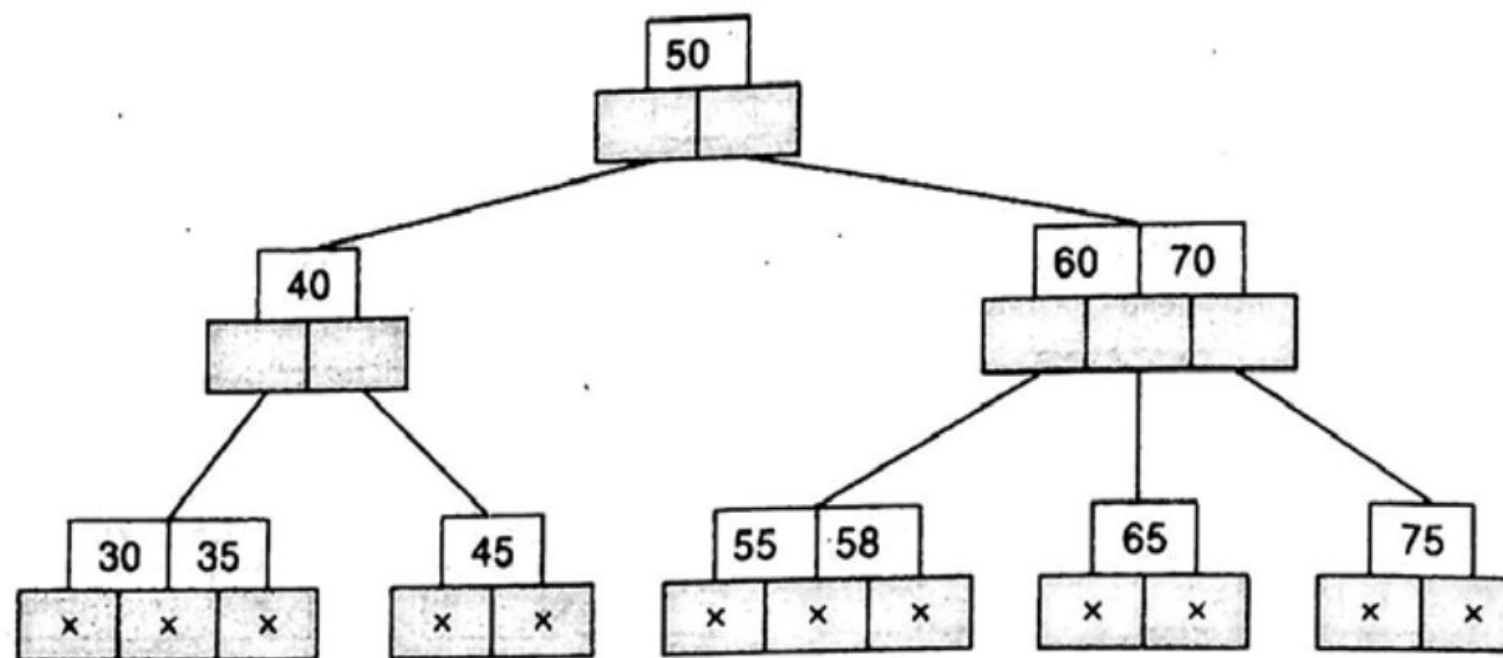
B-tree of order 3:



Deletion in B Trees [1]

Example: Insert 75, 57, deletion 35, 65

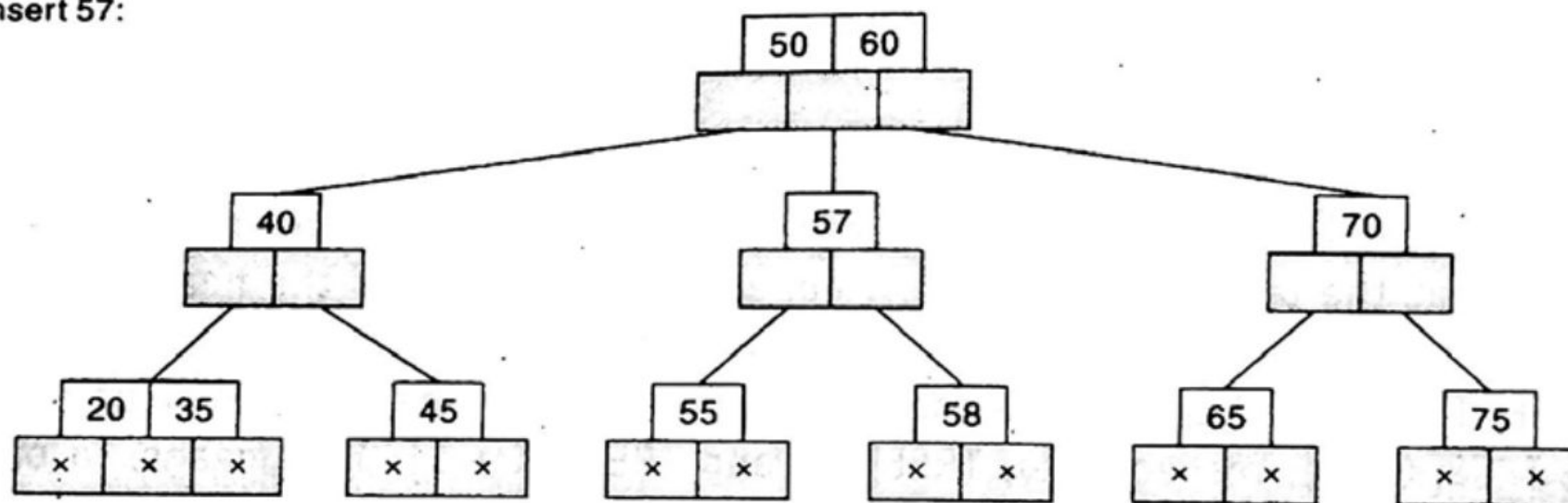
Insert 75:



Deletion in B Trees [1]

Example: Insert 75, 57, deletion 35, 65

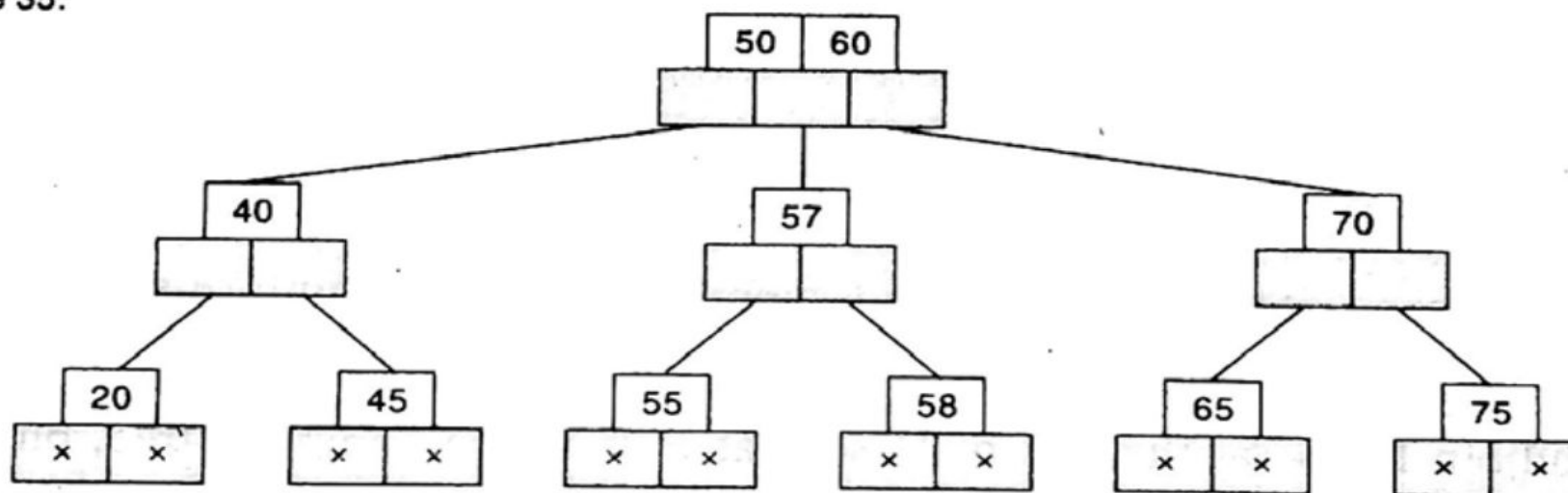
Insert 57:



Deletion in B Trees [1]

Example: Insert 75, 57, deletion 35, 65

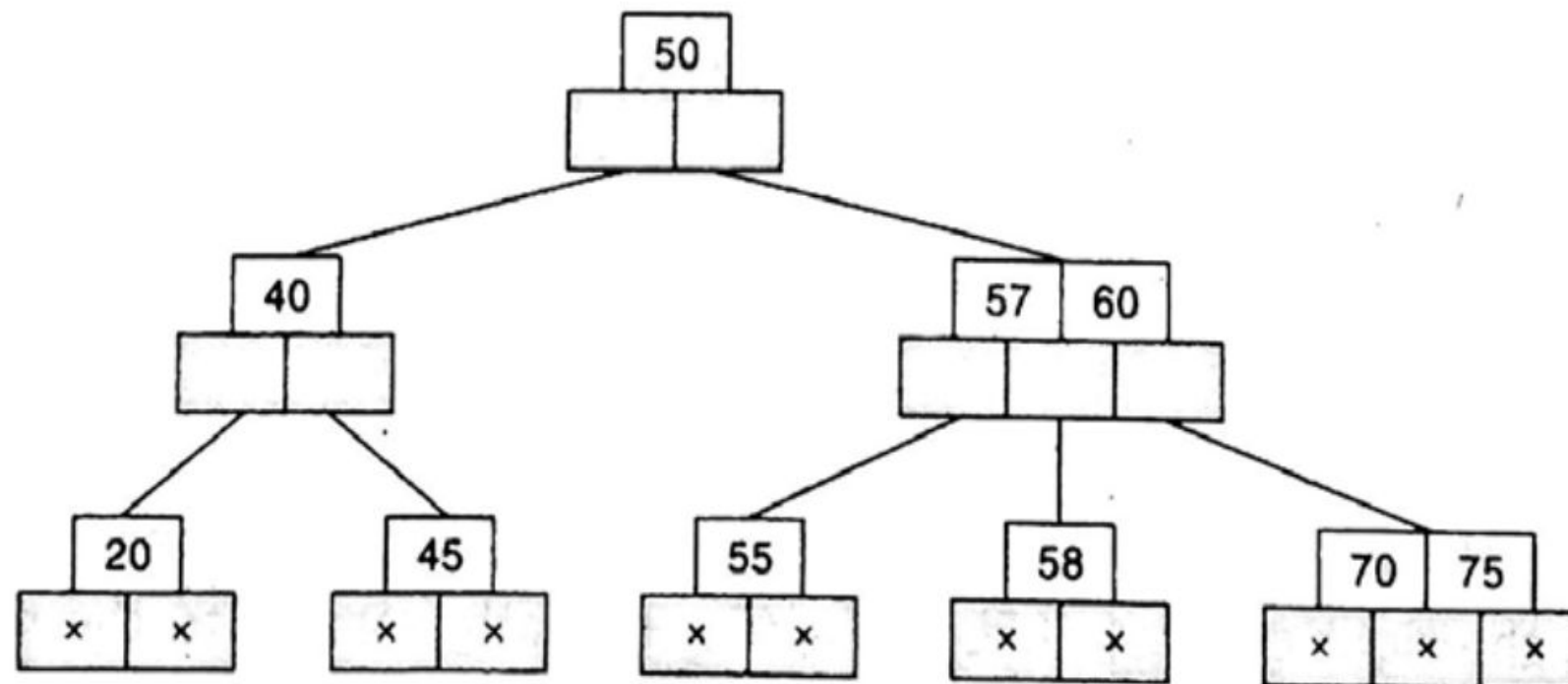
Delete 35:



Deletion in B Trees [1]

Example: Insert 75, 57, deletion 35, 65

Delete 65:





References

1. Seymour Lipschutz, “Data Structures”, Schaum’s Series McGraw Hill edition 2013.
2. <https://en.wikipedia.org/>
3. https://www.datadoctor.biz/data_recovery_programming_book_chapter2-page16.html